

AI 하는 사람을 위한 깃허브 초간단

AI 하는 사람을 위한 깃허브 초간단 — 목차

제품 포지셔닝

나두Ai의 『AI 하는 사람을 위한 깃허브 초간단』은 이미 AI 도구를 쓰는 분이 GitHub.com에서 **계정 → 저장소 → 클론 → 커밋 → 푸시 → Pull Request** 한 바퀴를 화면 따라 완주하게 돕는 전자책입니다. AI·코딩 입문은 가정하고, **GitHub가 낯선 지점만** 메웁니다. Git 심화·Actions·모노레포는 다루지 않습니다. 브라우저와 github.com이면 충분합니다. 나두Ai 영상·팁을 보고 “내 PC·계정에서 어떻게 하지?”에서 멈춘 분은 **부록 A** 주제별 실습으로 이어가면 됩니다. 본편 1~9장 제목·순서는 그대로입니다.

부록

영상 주제별 실습 → 부록 A · 트렌드 실습 → 부록 B (본편 제목·순서 유지).

무료 PDF: <https://tinalinkeom.vercel.app/ebook>

브랜드: 나두Ai

대상: AI 도구는 쓰지만 GitHub가 낯선 한국어 사용자

전체 목차

장	제목	예상 페이지
1장	이 책으로 할 수 있는 것 / 안 다루는 것	4~5p
2장	GitHub 계정 만들기	4~5p
3장	저장소(Repository) 이해하기	4~5p
4장	클론(Clone) — 내 컴퓨터로 복사해 오기	4~5p
5장	커밋(Commit) — 변경을 기록하기	4~5p

장	제목	예상 페이지
6장	푸시(Push) — GitHub에 올리기	3~4p
7장	Pull Request(PR) — 변경을 제안하기	4~5p
8장	자주 막히는 장면 Q&A	3~4p
9장	다음에 연습할 것	2~3p
부록	나두 인기 영상 실습 (Lab 1~8)	별도
부록	깃허브 트렌드 실습 (Lab 1~8)	별도
고지	고지·법적 주의 (별도 장/부록)	별도
—	합계	약 32~41p (+부록·고지)

1장. 이 책으로 할 수 있는 것 / 안 다루는 것

학습 목표 - 이 책의 끝에서 스스로 할 수 있는 일 다섯 가지를 말할 수 있다 - “다루는 것”과 “안 다루는 것”을 구분할 수 있다 - Git과 GitHub의 차이를 한 문장으로 설명할 수 있다 - 따라하기에 필요한 준비물(브라우저, 인터넷, 이메일)을 확인한다

예상 페이지: 4~5p

2장. GitHub 계정 만들기

학습 목표 - github.com에서 새 계정을 만들 수 있다 - 이메일 인증과 기본 프로필 설정을 끝낼 수 있다 - 로그인 / 로그아웃을 스스로 할 수 있다 - 비밀번호·2단계 인증(2FA)의 기본 의미를 안다 (강제 설정은 선택)

예상 페이지: 4~5p

3장. 저장소(Repository) 이해하기

학습 목표 - 저장소(Repository)가 “프로젝트 폴더의 온라인 버전”임을 이해한다 - Public / Private의 차이를 말할 수 있다 - 빈 저장소를 github.com에서 새로 만들 수 있다 - README가 무엇인지, 왜 커 두면 좋은지 안다

예상 페이지: 4~5p

4장. 클론(Clone) — 내 컴퓨터로 복사해 오기

학습 목표 - 클론 = “원격 저장소를 내 컴퓨터로 통째로 복사해 오기”임을 이해한다 - github.com에서 Clone URL(HTTPS)을 복사할 수 있다 - 브라우저 또는 GitHub Desktop으로 클론하는 흐름을 따라 한다 - 클론이 끝난 뒤 “어디 폴더에 생겼는지” 확인할 수 있다

예상 페이지: 4~5p

5장. 커밋(Commit) — 변경을 기록하기

학습 목표 - 커밋 = “이 시점의 변경을 한 묶음으로 기록하기”임을 이해한다 - 파일을 조금 고친 뒤, 변경을 스테이징(선택)하고 커밋할 수 있다 - 커밋 메시지를 짧고 분명하게 쓰는 감을 익힌다 - “커밋만 하고 아직 GitHub에는 안 올린 상태”를 구분한다

예상 페이지: 4~5p

6장. 푸시(Push) — GitHub에 올리기

학습 목표 - 푸시 = “내 컴퓨터의 커밋을 GitHub 원격 저장소에 보내기”임을 이해한다 - 푸시 후 github.com에서 파일이 바뀐 것을 확인할 수 있다 - 푸시가 실패했을 때 먼저 볼 곳(로그인·권한·인터넷)을 안다 - 클론 → 커밋 → 푸시의 한 바퀴를 말로 설명할 수 있다

예상 페이지: 3~4p

7장. Pull Request(PR) — 변경을 제안하기

학습 목표 - PR = “내 변경을 본 저장소(또는 기본 브랜치)에 합쳐 달라고 제안하는 글”임을 이해한다 - 간단한 브랜치를 만들고, 그 위에서 커밋·푸시한 뒤 PR을 연다 - PR 제목·설명을 초보 수준으로 작성할 수 있다 - Merge(병합)의 의미를 “제안이 받아들여져 합쳐진 것”으로 이해한다

예상 페이지: 4~5p

8장. 자주 막히는 장면 Q&A

학습 목표 - 로그인·인증·권한 관련 흔한 막힘을 스스로 점검할 수 있다 - “어디에 올렸는지 모르겠다” 상황을 되짚는 순서를 안다 - 클론/푸시/PR에서 자주 보는 메시지를 두려워하지 않는다 - 이 책 범위를 넘는 문제일 때 “다음에 무엇을 찾아볼지” 방향을 안다

예상 페이지: 3~4p

9장. 다음에 연습할 것

학습 목표 - 혼자 반복 연습할 미니 미션 3~5개를 고른다 - “한 바퀴 더” 체크리스트로 복습한다 - 이 책 다음에 배우면 좋은 주제(브랜치 이름 규칙, Issue 등)를 가볍게 안다 - Actions·심화 Git은 일부러 나중에 두기로 한다

예상 페이지: 2~3p

범위 요약 (집필·배포 공통)

다름	안 다름
계정, 저장소, 클론, 커밋, 푸시, PR	GitHub Actions
github.com 화면 따라하기	Git 심화(리베이스, 체리픽 등)
HTTPS 클론·푸시 기본	대규모 모노레포

다름	안 다름
GitHub Desktop 가볍게 언급(선택)	VS Code 심화 설정 필수화

나두Ai · 화면 따라하기용

고지 · 법적 주의 (별도 장 / 부록)

위치: 본편 1~9장 · 실습 부록과 **별도**. 장 번호 1~9는 바꾸지 않습니다.

파일:

내용 요약: 저작권(무료 학습 · 무단 재판매 금지) · 제3자 상표(비제휴) · OSS · 레포 · 나두Ai 영상과 PDF 관계 · 교육 면책 · 금지 안내 · 개인정보(최소) · **무료 제공(유료 · 환불 미적용)** · 배포자 표시

※ AI가 정리한 체크리스트이며 법률 의견이 아닙니다. 유료 재개 시 통신판매 · 세금은 사용자 · 전문가 확인이 필요합니다.

1장. 이 책으로 할 수 있는 것 / 안 다루는 것

AI 도구로 코드·글을 받아 본 적이 있다면, 그 감각이면 충분합니다.

이 책이 메우는 빈칸은 **GitHub에 올리고 제안하는 한 바퀴**뿐입니다.

github.com 화면 문구는 가끔 바뀝니다. 버튼 글자가 조금 달라도 **비슷한 문구**면 그 버튼을 누르면 됩니다. (이 안내는 이 책에서 여기만 적습니다.)

1. 끝까지 따라가면 생기는 일

1. **GitHub 계정**을 만들고 로그인한다
2. **저장소(Repository)** 를 하나 만든다
3. **클론(Clone)** 해서 내 컴퓨터(또는 Desktop이 보는 폴더)로 가져온다
4. 파일을 고친 뒤 **커밋(Commit)** → **푸시(Push)** 로 github.com에 반영한다
5. **Pull Request(PR)** 로 “합쳐 주세요”를 제안해 본다

계정 → 저장소 → 클론 → 커밋 → 푸시 → PR

이 한 바퀴를 한 번 완주한다.

화면 참고: 책 전체 흐름 — 계정 → 저장소 → 클론 → 커밋 → 푸시 → PR

2. 다루는 것

주제	의미	어디서
계정	github.com 가입·로그인	브라우저
저장소	온라인 프로젝트 폴더 만들기	github.com
클론	원격 저장소를 내쪽으로 복사	github.com + (선택) Desktop

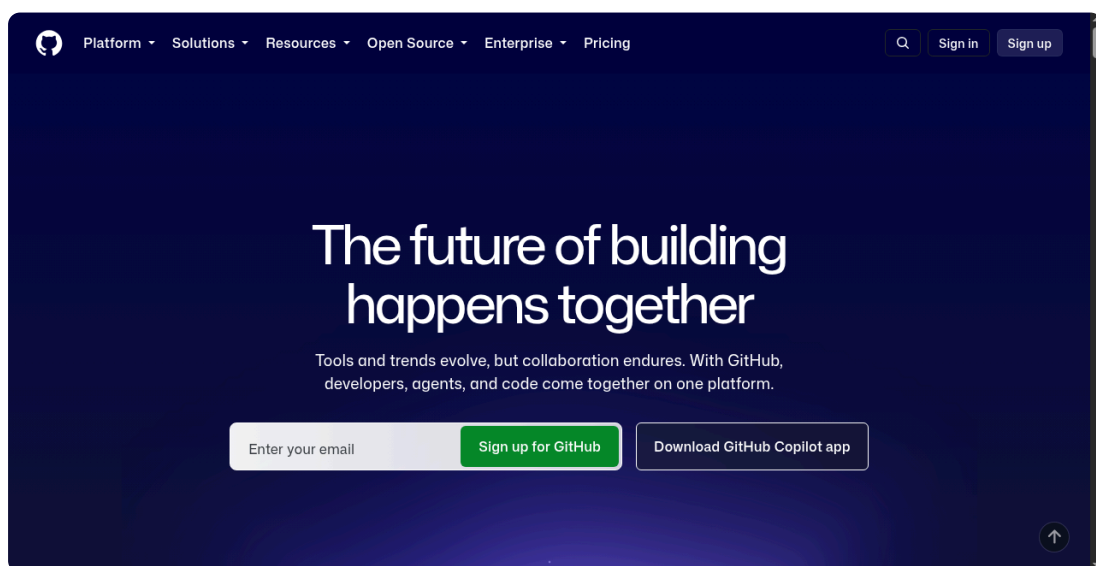
주제	의미	어디서
커밋	변경을 한 묶음으로 기록	Desktop 등
푸시	커밋을 GitHub로 보내기	위와 동일
PR	합쳐 달라는 제안 글	github.com

용어 한 줄

- **GitHub** — 코드를 올리고 보는 웹 서비스 (<https://github.com>)
- **Git** — 변경 이력을 기록하는 도구. GitHub가 그 이력을 웹에서 보여 줌
- **저장소(Repository, repo)** — 프로젝트의 온라인 폴더
- **클론** — 온라인 폴더를 내 컴퓨터로 통째로 복사
- **커밋** — “여기까지 바꿨다” 기록 (아직 웹에 자동 업로드되지 않음)
- **푸시** — 쌓인 커밋을 GitHub로 보내기
- **PR** — 기본 줄기에 합쳐 달라는 제안서

Git ≠ GitHub. Git은 기록 도구, GitHub는 올려 두고 보는 웹.

이 책은 **GitHub 화면 따라가기**가 중심이고, Git 명령어 심화는 안 합니다.



github.com 첫 화면

3. 안 다루는 것

안 다룸	이유
GitHub Actions	자동 빌드·배포 — 계정·PR 다음에
Git 심화	리베이스, 체리픽 등
대규모 모노레포	권한·규칙이 다름
VS Code 심화	에디터 튜닝은 범위 밖
유료 팀·조직 관리	Organization 고급·결제

지금은 **한 바퀴 완주**가 목표입니다. Actions까지 할 필요 없습니다.

4. 누가 읽으면 맞나

- AI로 파일을 받아 봤는데, GitHub에 올리는 법이 막힌다
- 클론·푸시·PR을 검색해 본 적이 있다
- 혼자라도 **내 저장소에 올리고 PR까지** 연습하고 싶다

이미 매일 PR·Actions를 쓰는 분에게는 복습용 수준입니다.

5. 준비물

꼭 필요: 인터넷 · 웹 브라우저 · 이메일 · 따라할 시간(장마다 나뉘도 됨)

있으면 편함(선택): GitHub Desktop(클론·커밋·푸시 버튼화), 메모장/텍스트 에디터

기본 경로: 브라우저 + github.com · Desktop은 선택 · 터미널 Git·VS Code Git 심화는 제외

화면 참고: 준비물 — 인터넷 / 브라우저 / 이메일 / (선택) Desktop

6. 읽는 법

1. 번호 매긴 단계를 그대로 따른다
2. 「여기서 막히면」 은 막혔을 때만 보면 됨

연습용은 **본인이 만든 빈 저장소·README만 있는 저장소**에서 하세요.

중요한 회사·님의 저장소에는 바로 적용하지 마세요.

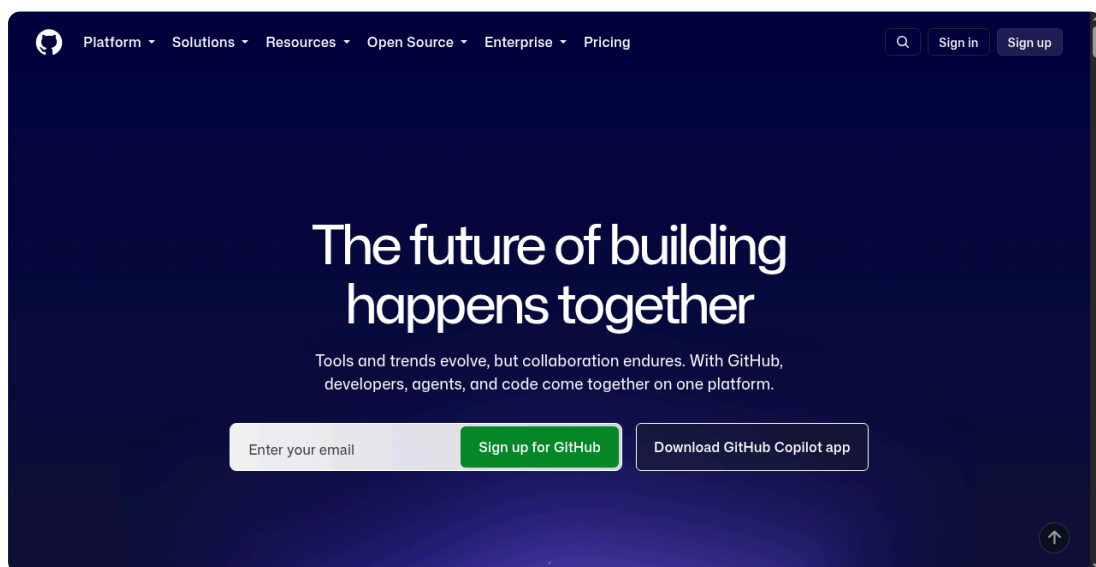
여기서 막히면

“코드가 망가질까 봐 무섭다” → 연습용 저장소를 **새로** 만드세요.

7. 영어 버튼 (그 장에서 누르는 것만)

- **Sign up** = 가입 · **New repository** = 새 저장소 · **Clone** = 복사해 오기
- **Commit** = 기록 · **Push** = 올리기 · **Pull request** = 합쳐 달라는 제안

전부 외울 필요 없습니다.



github.com 상단 영역

8. 안전 (짧게)

- 비밀번호·토큰·복구 코드는 책·채팅·캡처에 적지 마세요. (이후 장에서 반복하지 않습니다.)
 - 연습은 본인 계정·본인 저장소에서. 남의 저장소에 함부로 푸시·PR하지 마세요.
 - 이 책은 법률·세금·결제 안내가 아닙니다.
-

다음

장	할 일
2장	계정 만들기
3장	저장소 만들기
4~6장	클론 → 커밋 → 푸시
7장	PR
8~9장	막힘 Q&A · 혼자 연습

계정이 이미 있으면 1장의 범위만 확인하고 3장부터 가도 됩니다.

여기서 막히면

맥/윈도우 모두 OK(브라우저면 2장까지 차이 거의 없음). 코딩 몰라도 README 한 줄이면 연습 가능.

2장. GitHub 계정 만들기

브라우저만 열고 `https://github.com` 에서 **계정 만들기** → **이메일 인증** → **로그인**·**로그아웃**까지 합니다.

VS Code·Desktop은 이 장에서 필요 없습니다.

학습 목표

- 새 계정을 만든다 · 이메일 인증 · 기본 프로필을 끝낸다
 - 로그인/로그아웃을 스스로 한다
 - 비밀번호·2FA의 의미를 안다 (2FA 설정은 선택)
-

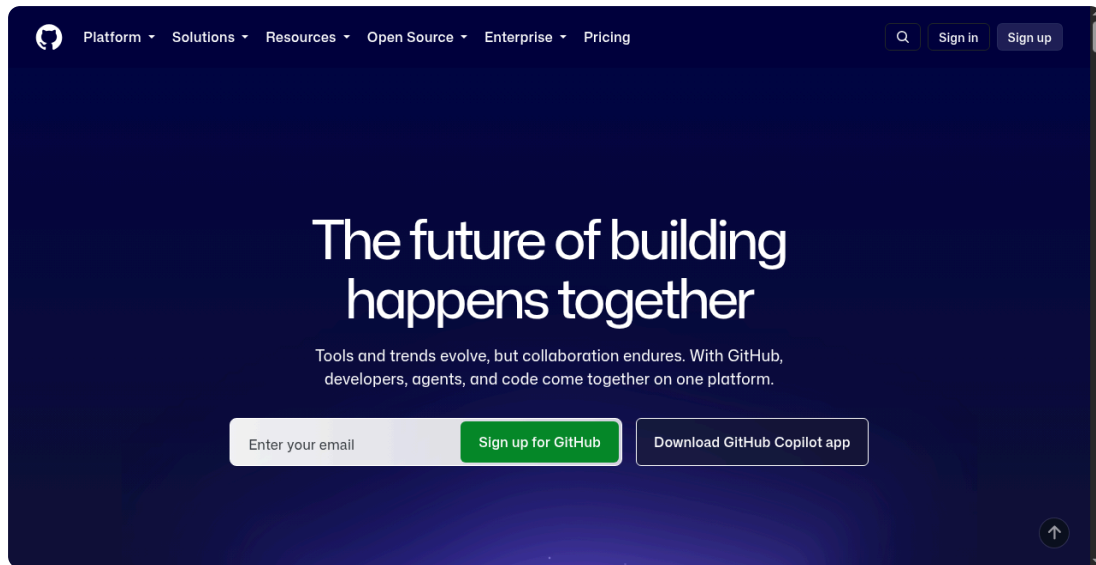
1. 시작 전

1. 인터넷 · 웹 브라우저 · 열어 볼 수 있는 **이메일**
2. 비밀번호는 **안전한 곳**(비밀번호 관리 앱·종이 등)에만 두세요. **책·채팅·캡처에 적지 마세요.**

이미 계정이 있으면 가입(2~4절)은 건너뛰고 **5~7절**만 하면 됩니다.

여기서 막히면

예전에 가입한 것 같으면 새 가입 전에 **Sign in** → **Forgot password?** 로 이메일 재설정을 먼저 시도하세요.



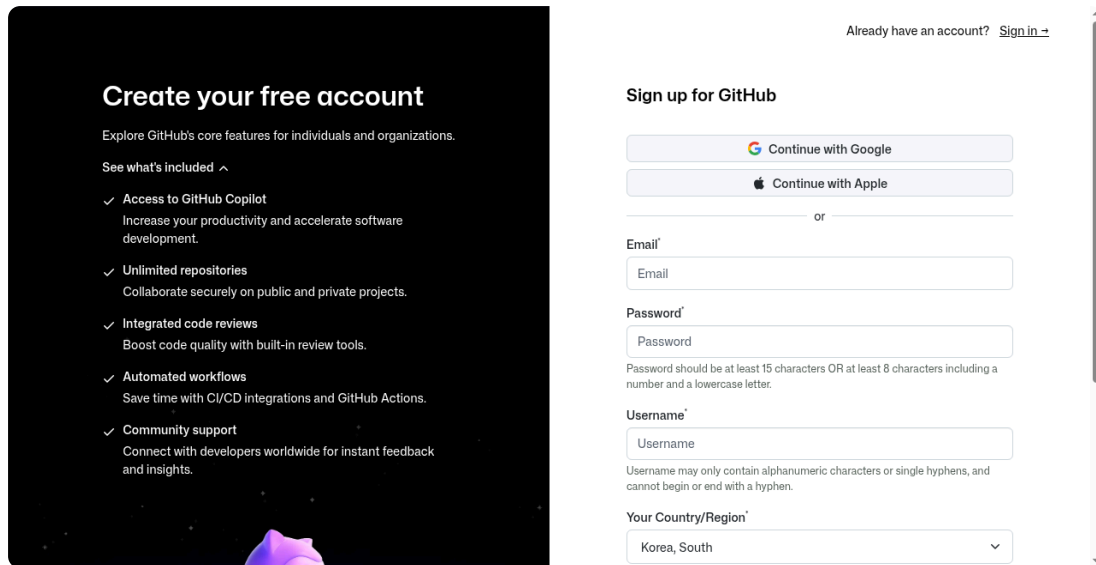
github.com — Sign up / Sign in 부근

github.com — Sign up / Sign in 부근

2. Sign up

1. `https://github.com` (또는 `https://github.com/signup`)
2. **Sign up** 클릭
3. 개인 **Free** 계정 기준. Pro·Team 안내는 지금은 무시해도 됩니다.

회사 메일 필수는 아닙니다. **임시(일회용) 메일**은 인증이 막힐 수 있어 쓰지 마세요.



Sign up 화면

Sign up 화면

3. 이메일 · 비밀번호 · Username

화면이 한 페이지이든 Continue로 나뉘든, 묻는 내용만 같으면 됩니다.

이메일 — 지금 열어 볼 주소. 오타 확인.

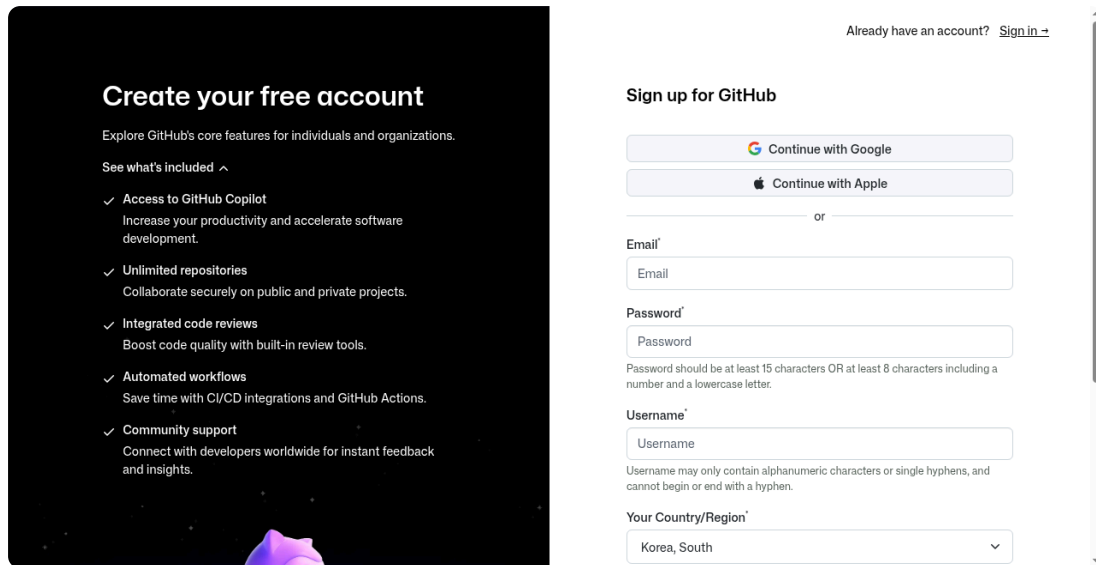
비밀번호 — 대략 15자 이상, 또는 8자 이상 + 숫자·소문자. (안내 문구는 바뀔 수 있음)
다른 사이트와 같은 비밀번호 재사용은 피하세요.

Username — 주소의 일부 (`https://github.com/여기에-이름`).

영문·숫자·하이픈(-), 앞뒤 하이픈은 피함. 이미 쓰이면 짧은 단어·숫자를 붙이세요.

표시 이름(Name)은 나중에 프로필에서 다르게 둘 수 있습니다.

국가/지역·약관·Captcha — 거주 국가 선택, 약관 한 번 훑기, 사람 확인 후 **Create account / Continue.**



가입 입력란(값은 가린 예시 화면)

가입 입력란(값은 가린 예시 화면)

여기서 막히면

Captcha가 반복 실패하면 광고 차단을 끄거나 다른 브라우저로 signup을 여세요.

4. 이메일 인증 (Verify)

인증이 안 끝나면 저장소 만들기 등이 막힐 수 있습니다. 이 장에서 끝냅니다.

1. 가입 메일함에서 GitHub 메일(verify / confirm / launch code 등) 찾기
2. 링크 또는 코드로 인증
3. github.com으로 돌아오면 성공에 가깝습니다

메일이 없으면: 스팸함 → 주소 오타 → **Resend** → 링크 만료 시 재발송.

로그인만 됐고 인증만 남았으면: 아바타 → **Settings** → **Emails** → Resend verification.

화면 참고: “이메일을 확인하세요” 안내 — 주소 가림

5. 로그인 확인

로그인되면 오른쪽 위 **아바타**, Sign up 대신 알림 · + · 아바타 메뉴가 보입니다.

플렀다면: <https://github.com/login> → **Sign in** → Username(또는 이메일) + 비밀번호.

잊었으면 **Forgot password?**

화면 참고: 로그인 후 상단 바 — 아바타(실명·얼굴 없이)

6. 프로필 (최소)

얼굴·실명 필수는 아닙니다.

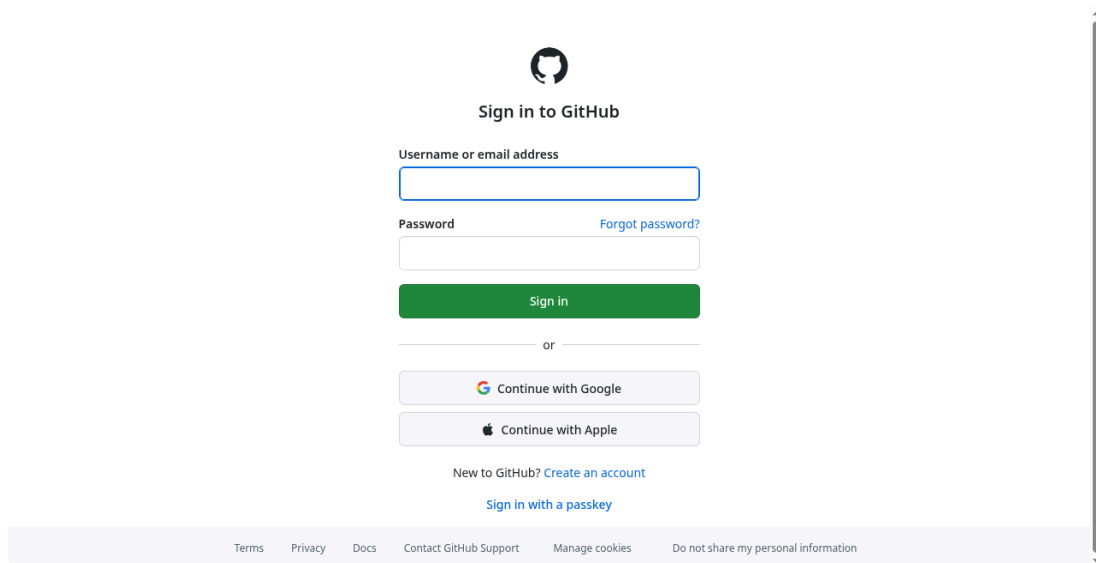
1. 아바타 → **Your profile** 또는 **Settings**
2. Avatar / Name / Bio는 선택. Email이 인증·Primary인지만 확인
3. Save / Update profile

화면 참고: Settings → Profile

7. Sign out → Sign in 연습

1. 아바타 → **Sign out**
2. 다시 **Sign in**

공용 PC에서는 실습 후 반드시 Sign out.



로그인 화면(Sign in) — 로그아웃 후 다시 들어올 때

로그인 화면(Sign in) — 로그아웃 후 다시 들어올 때

8. 비밀번호 · 2FA (의미만)

비밀번호 — github.com 열쇠. (어디에 적어도 되는지는 위 1절.)

2FA — 비밀번호 외 두 번째 확인(앱 코드 등). 커면 유리하지만 **이 책에서 강제하지 않습니다.**

가입 직후 마법사가 뜨면 공식 단계를 따르거나, Skip/나중예가 허용되면 한 바퀴를 먼저 완주해도 됩니다.

복구 코드가 보이면 안전한 곳에만 두세요.

이 장에서 안 함

VS Code · Desktop · 터미널 `git` · 유료·Organization · 얼굴·실명 공개 강제

기본 경로: 브라우저 + <https://github.com>

짧게 정리

계정 생성 → **이메일 인증** → 프로필은 최소 → Sign out/in 한 번 → 2FA는 선택.

다음 **3장**에서 연습용 **저장소**를 만듭니다. Create a repository 안내가 보여도 지금은 급히 만들지 말고 3장에서 합니다.

화면 참고: 3장 예고 — New repository 버튼 부근

나두Ai · 『AI 하는 사람을 위한 깃허브 초간단』

3장. 저장소(Repository) 이해하기

로그인된 상태에서 **저장소**가 무엇인지 보고, **연습용 저장소 하나**를 만듭니다.
클론·커밋·푸시는 아직 하지 않습니다. VS Code·Desktop도 이 장에서 불필요합니다.

학습 목표

- 저장소 = 프로젝트 폴더의 온라인 버전
- Public / Private 차이를 말할 수 있다
- 빈 저장소(+ README)를 만든다 · README가 뭔지 안다

1. 저장소란?

저장소(Repository, repo) = 프로젝트 폴더의 온라인 버전

내 컴퓨터	GitHub
폴더	저장소
폴더 이름	저장소 이름
폴더 안 파일	저장소 안 파일

한 바퀴 중 지금은 「**저장소**」 : 계정 → **저장소** → 클론 → 커밋 → 푸시 → PR

화면 참고: 저장소 = 온라인 폴더 비유

리포지토리·레포·repository는 **같은 말**입니다. 이 책은 **저장소**를 기본으로 씁니다.

2. Public / Private

종류	의미	연습
Public	계정이 있는 사람이 내용을 볼 수 있는 편	공개 연습·포트폴리오
Private	본인(과 초대한 사람) 위주	확신이 없으면 추천

비밀번호·개인키·고객 데이터는 Public/Private와 관계없이 **올리지 마세요.**

조직(Organization) 복잡한 권한은 다루지 않습니다. 지금은 본인 계정에서 하나만 고르면 됩니다.

화면 참고: *New repository — Public / Private*

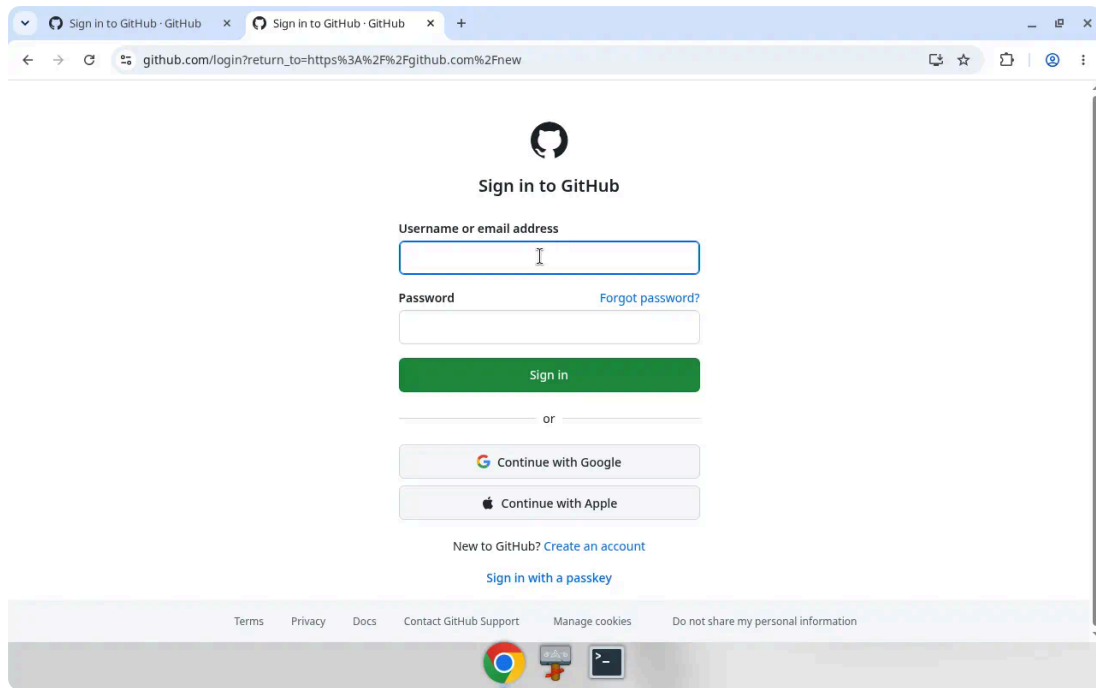
나중에 공개 범위는 바꿀 수 있는 경우가 많습니다. 다만 한 번 Public에 올린 내용은 이미 본 사람·복사본이 있을 수 있습니다.

3. New repository

1. `https://github.com` — 로그인·아바타 확인
2. 오른쪽 위 + → **New repository**
또는 `https://github.com/new`

여기서 막히면

+가 안 보이면 로그인·창 너비·`/new` 직접 이동. 이메일 미인증이면 2장 Verify를 먼저.



New repository — 로그인 필요 시 이 화면으로 안내될 수 있음

New repository — 로그인 필요 시 이 화면으로 안내될 수 있음

4. 이름

연습용 예: `hello-github` · `my-first-repo` · `practice-readme`

- 영문·숫자·하이픈(-)·밑줄(_), 공백은 피함
- 같은 계정 안 이름 중복 불가 · 초보는 소문자+하이픈이면 충분
- Description은 비우거나 “연습용” 한 줄 · Owner는 **본인 Username**

화면 참고: *Repository name — hello-github 예시*

이름이 중복이면 `hello-github-2` 처럼 조금 바꾸세요.

5. Public/Private + README ON

- 연습·확신이 없으면 → **Private**

- **Add a README = ON** (권장)

README — 저장소를 열면 먼저 보이는 안내 파일(README.md). 표지에 가깝습니다.
켜 두면 빈 화면이 덜하고, 다음 장 클론·5장 커밋 연습에 유리합니다.

.gitignore·license는 끄거나 기본값·None이면 됩니다. 이 장에서 안 다룹니다.

화면 참고: Public/Private + Add a README ON

README를 안 켜다면 빈 화면의 “create a README”를 따르거나, README ON으로 새 연습 저장소를 만들어도 됩니다.

6. Create repository

Create repository 클릭 → 새 저장소 페이지로 이동하면 성공.

주소: `https://github.com/내Username/저장소이름`

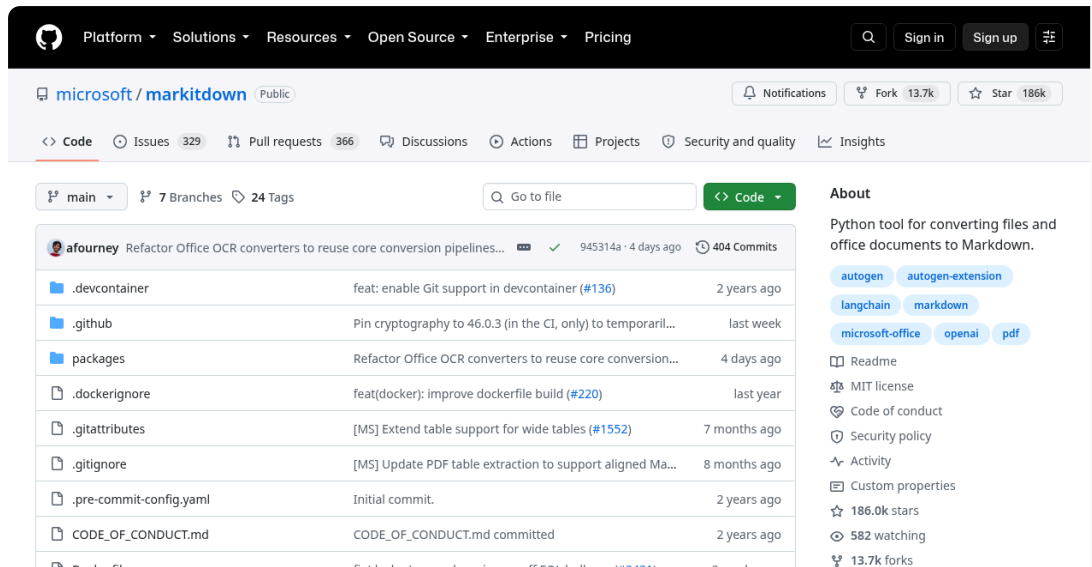
버튼이 비활성이면 이름 칸 아래 빨간 안내(비움·규칙·중복)를 고칩니다.

화면 참고: Create repository 버튼

7. 만들어진 화면 둘러보기

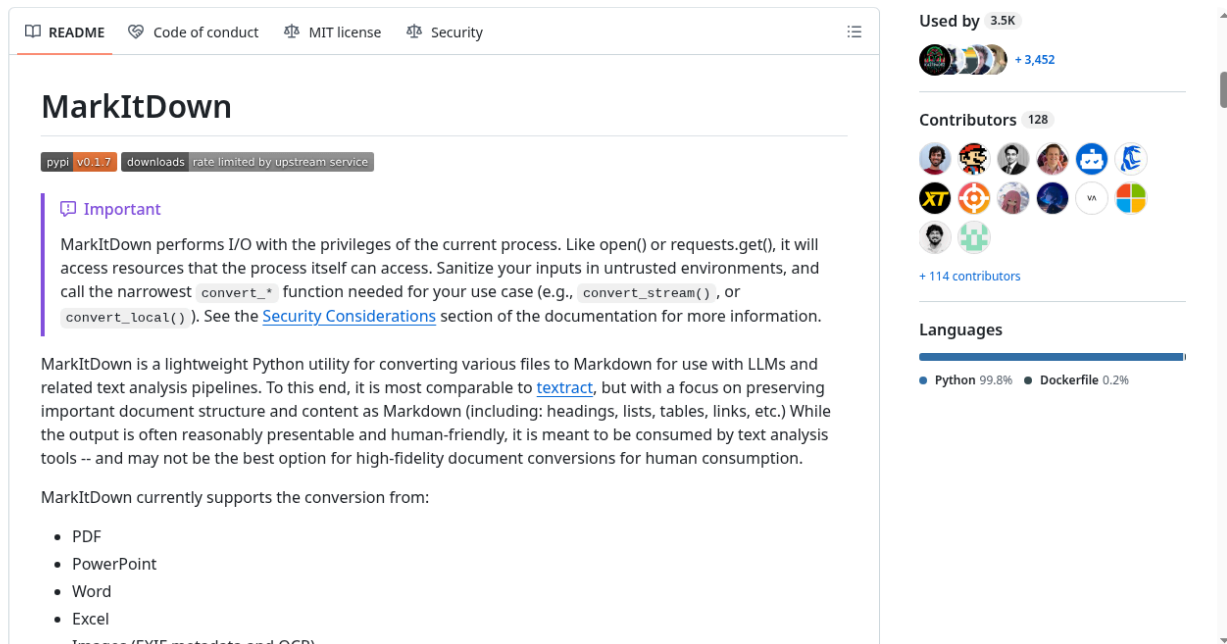
구경만 합니다. 클론은 4장.

1. 저장소 이름 · Public/Private · **Code** 탭
2. README.md 미리보기 — “표지가 생겼구나”만 확인
3. 초록 **Code** 버튼 위치만 눈으로 확인 (HTTPS·Desktop 메뉴 — **아직 클론하지 않음**)
4. 파일이 한두 개·Initial commit이 보여도 정상. 커밋 심화는 5장.

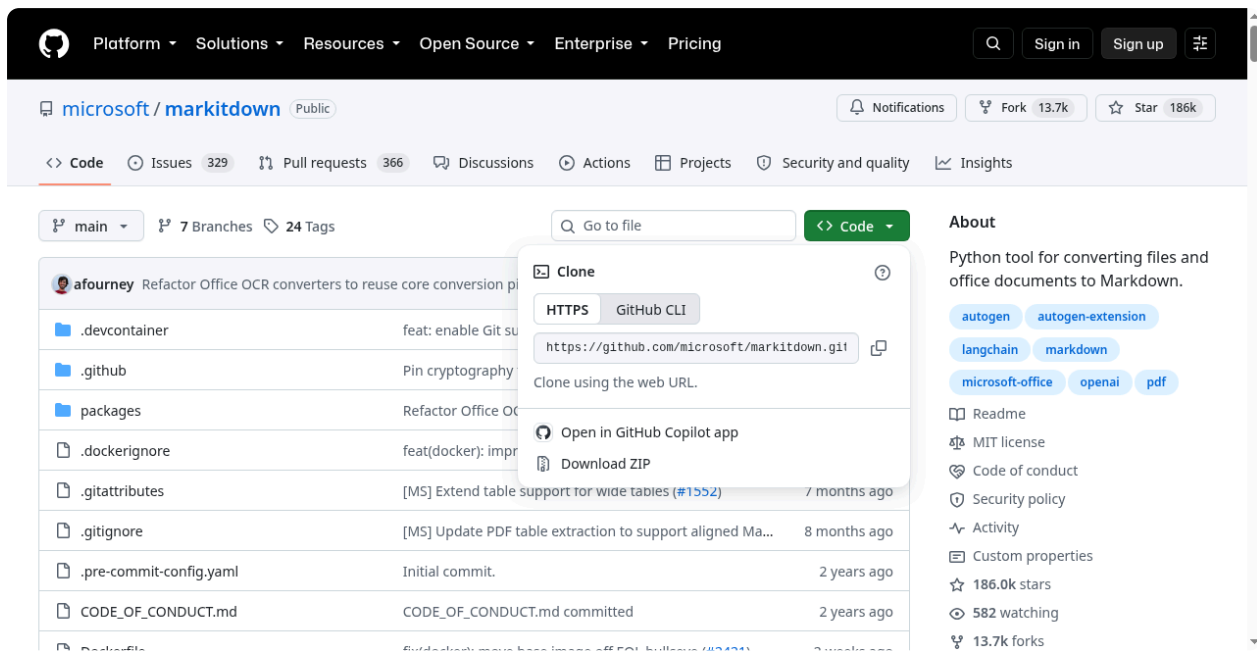


저장소 홈 예시(공개 저장소 화면)

저장소 홈 예시(공개 저장소 화면)



README 미리보기 예시



Code 드롭다운 — HTTPS Clone URL

이 장에서 안 함

클론(4장) · 커밋·푸시(5~6) · PR(7) · 이름 변경 · 삭제 심화 · Actions · .gitignore·라이선스

짧게 정리

저장소 = 온라인 폴더 · 연습은 확신이 없으면 Private · **README ON** · Create 후
Code·README만 확인 · **클론은 4장**.

화면 참고: 4장 예고 — Code 버튼과 HTTPS Clone URL

나두Ai · 『AI 하는 사람을 위한 깃허브 초간단』

4장. 클론(Clone) — 내 컴퓨터로 복사해 오기

3장 저장소(README 포함)를 **클론**해 내 컴퓨터 폴더로 가져옵니다.

기본 경로는 **GitHub Desktop**. 커밋·푸시는 5~6장.

학습 목표

- 클론 = 원격 저장소를 내 컴퓨터로 통째로 복사
- HTTPS Clone URL 복사 · Desktop으로 클론 · 폴더 위치 확인

1. 클론이란?

클론 = GitHub 저장소를 내 컴퓨터로 통째로 복사해 오기

GitHub (원격)	클론 후 (로컬)
<code>github.com/내Username/저장소이름</code>	예: <code>Documents/GitHub/저장소이름</code>
README가 웹에 보임	같은 파일이 탐색기에도 있음
	Git 이력이 함께 복사됨 (ZIP만 받는 것과 다름)

한 바퀴: 계정 → 저장소 → **클론** → 커밋 → 푸시 → PR

화면 참고: 클론 비유 — github.com → PC 폴더

2. ZIP ≠ 클론

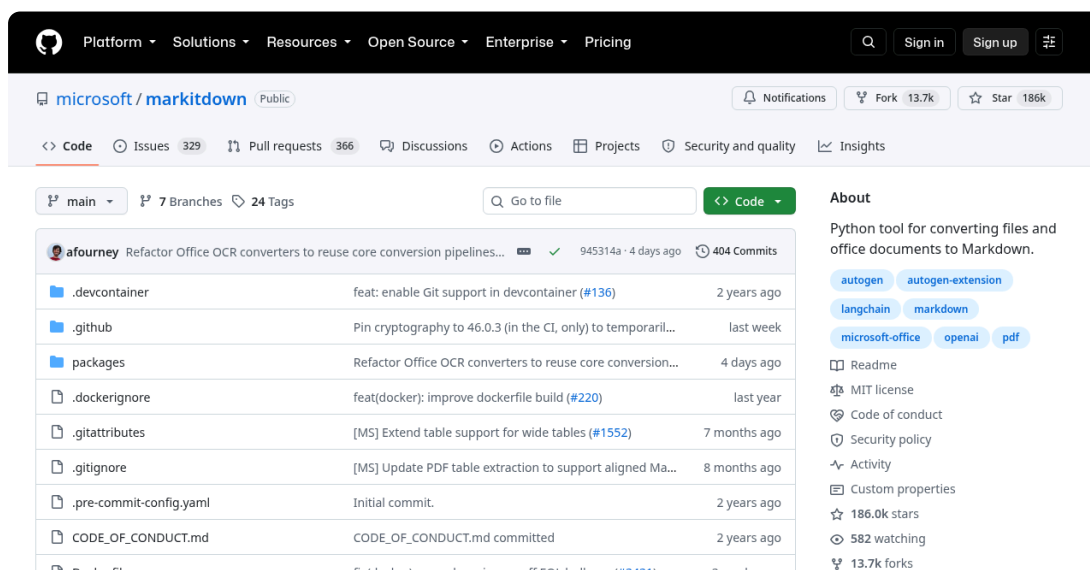
	Download ZIP	클론
무엇	파일 압축본	Git과 함께 저장소 복사
이후 커밋 · 푸시	이 책 경로로 준비 안 됨에 가까움	Desktop 등으로 이어가기 좋음
권장	구경·비상용	이 장 기본

한 바퀴(클론→커밋→푸시)에는 **클론**. 이 장에서 ZIP은 누르지 않아도 됩니다.

3. 시작 전

- 계정 로그인(2장) · README 있는 저장소(3장) · 인터넷
- (기본) Desktop 설치 가능 — 막히면 7절 참고. 이 책은 Desktop 기준

주소 형태: `https://github.com/내Username/저장소이름`



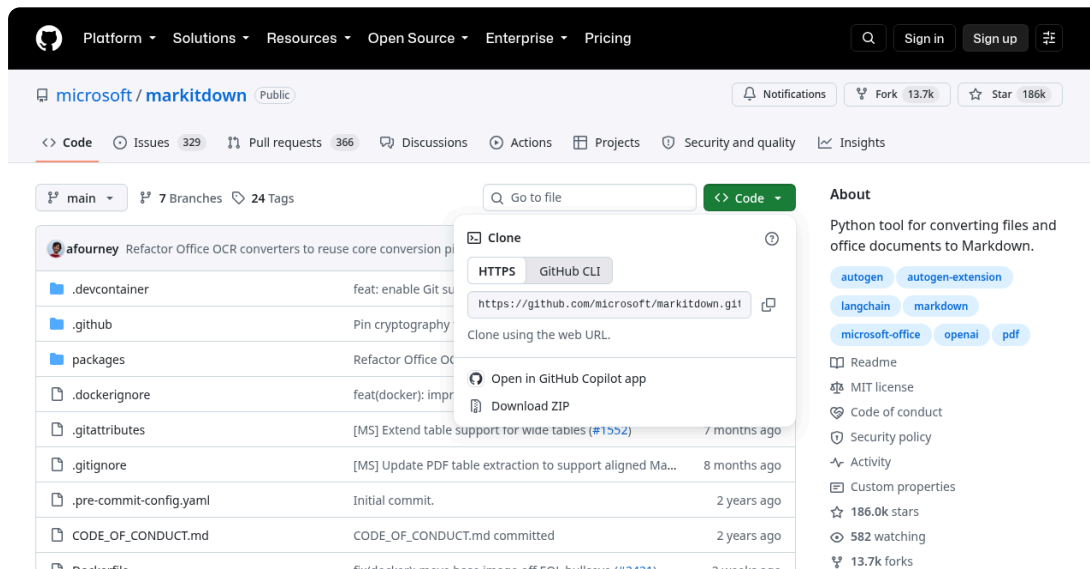
저장소 홈 — Code 탭

저장소·README가 없으면 3장으로 돌아가 README ON으로 만드세요.

4. Code → HTTPS 복사

1. 본인 저장소 페이지
2. **Code** → **HTTPS** (SSH는 이 장에서 안 함)
3. URL 옆 **Copy** — `https://github.com/내Username/저장소이름.git`

저장소 URL만이면 됩니다. (토큰·비밀번호는 적지 마세요 — 1·2장 안내와 동일)



Code — HTTPS URL 복사

Code — HTTPS URL 복사

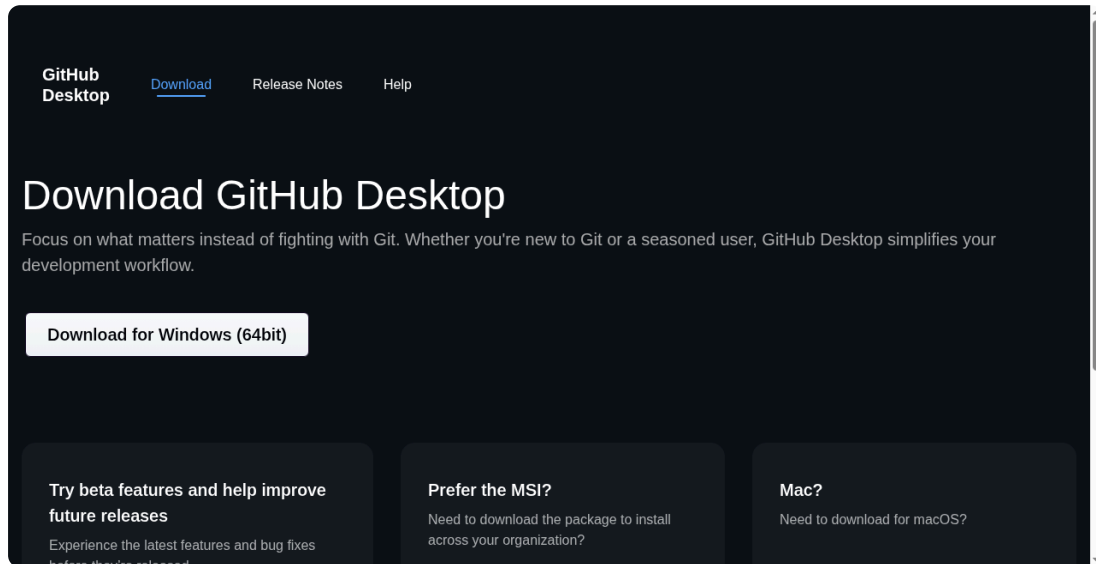
Code 탭(파일 목록)인지 확인하세요. Settings 등 다른 탭이면 Code로 돌아갑니다.

5. Desktop으로 클론

5-1. 설치 (없다면)

공식 **GitHub Desktop** 다운로드 → OS별 설치 → 실행.

이미 있으면 5-2로.



GitHub Desktop 다운로드 페이지

GitHub Desktop 다운로드 페이지

설치 권한이 없으면 권한이 있는 PC에서 하거나 7절을 참고만 하세요.

5-2. Sign in

1. **Sign in to GitHub.com**
2. 브라우저에서 **2장 계정**으로 로그인·Authorize
3. Desktop에 Username이 보이면 OK

5-3. Clone — A 또는 B 중 하나

A. File → **Clone repository**... → GitHub.com 탭에서 3장 저장소 선택 → **Local path** 확인
(예: Documents/GitHub) → **Clone**

B. 웹 Code → **Open with GitHub Desktop** → Local path 확인 → **Clone**

끝나면 Desktop에 저장소가 열리면 1차 성공입니다.

화면 참고: Desktop Clone 창 — 저장소 + Local path + Clone

여기서 막히면

목록에 없으면 같은 계정인지 확인하거나 URL 탭에 `https://...git` 붙여넣기.

“이미 있다”면 Local path를 다른 빈 위치로 바꾸거나 기존 폴더를 엽니다. (중요 파일은 지우지 마세요.)

6. 폴더 위치 확인

1. Desktop **Repository** → **Show in Explorer** / **Show in Finder**
2. 폴더 이름 = 저장소 이름 · 안에 **README.md**
3. (선택) README를 열어 내용만 확인 — 수정·커밋은 5장

경로 예(본인 Local path가 정답):

Windows `...\Documents\GitHub\hello-github` · macOS `.../Documents/GitHub/hello-github`

성공 기준: Desktop에 저장소 열림 · 로컬 경로를 앞 · README.md 있음

화면 참고: 탐색기 — 클론 폴더 안 README.md

README가 없으면: 웹에 README 있는지 · 클론한 저장소 이름이 맞는지 확인.

지웠다면 다시 클론하면 됩니다(웹 원본은 보통 그대로).

7. (참고) 터미널 — 필수 아님

```
git clone https://github.com/내Username/저장소이름.git
```

Git이 설치된 환경에서만. 인증·PATH 심화는 범위 밖. 막히면 Desktop(5~6절)으로.

이 장에서 안 함

커밋(5) · 푸시(6) · PR(7) · SSH 키 · ZIP만으로 이어가기 · Actions · VS Code 심화

기본 경로: **Code(HTTPS) + Desktop 클론 + 로컬 폴더 확인**

짧게 정리

클론 = 통째 복사 · **ZIP ≠ 클론** · HTTPS 복사 · Desktop 로그인 후 Clone · Local path + README 확인 · 커밋·푸시는 다음 장.

Private도 본인 계정 Desktop이면 되는 경우가 많습니다. 브라우저만으로는 “내 PC에 Git 연결까지”가 어렵습니다.

화면 참고: 5장 예고 — *Desktop Changes / Commit message*

나두Ai · 『AI 하는 사람을 위한 깃허브 초간단』

5장. 커밋(Commit) — 변경을 기록하기

4장에서 클론한 폴더의 `README.md` 를 한두 줄 고친 뒤, Desktop에서 커밋까지 합니다.

푸시는 하지 않습니다. 커밋만 하면 내 PC 기록에만 남고 github.com에는 아직 없습니다. 올리는 것은 6장.

학습 목표

- 커밋 = 이 시점의 변경을 한 묶음으로 기록
- 수정 → 스테이징(체크) → 커밋
- 짧은 메시지 · 커밋 ≠ 웹에 올라감(푸시는 6장)

1. 커밋이란?

커밋 = 이 시점의 변경을 한 묶음으로 기록하기

파일을 고쳤다고 자동 기록되지 않습니다. Commit을 눌러야 이력이 생깁니다.

단계	어디에	이 장
파일 수정·저장	내 PC 폴더	✓
커밋	내 PC Git 기록	✓
푸시	github.com	✗ → 6장

한 바퀴: 계정 → 저장소 → 클론 → 커밋 → 푸시 → PR

화면 참고: 커밋 비유 — PC 변경 → Commit → (아직) github.com 그대로

3장에서 README를 켜면 **Initial commit**이 이미 있을 수 있습니다. 이 장에서는 **직접** 다음 커밋을 만듭니다.

2. 스테이징 (초보용)

Desktop **Changes**에서 파일 옆 **체크**가 스테이징에 가깝습니다.

스테이징 = 이번 커밋에 넣을 파일 고르기(체크)

연습은 README 하나만 보이면 **체크 ON**으로 Commit하면 됩니다. `git add` 는 필수가 아닙니다.

화면 참고: Desktop Changes — README.md 체크 ON

3. 시작 전

Desktop에 4장 클론 저장소가 열려 있고, 폴더에 `README.md` , 같은 계정으로 로그인되어 있으면 됩니다.

없으면 4장으로 돌아가 클론하세요.

화면 참고: Desktop — 클론 저장소 선택, 변경 없음

4. README 한 줄 고치기

1. Repository → Show in Explorer / Finder
2. `README.md` 를 메모장·기본 편집기로 연다 (VS Code 심화 불필요)
3. 맨 아래에 한 줄 추가 예:

연습: 5장에서 커밋해 보기

4. 저장 (Ctrl+S / Cmd+S)

화면 참고: 편집기 — README.md 한 줄 추가·저장

여기서 막히면

Desktop에 Changes가 없으면: 저장했는지 · **클론한 그 폴더인지** · ZIP 폴더를 고친 건 아닌지 확인.

5. Changes 확인

1. Desktop → **Changes**에 README.md
2. (선택) diff로 추가 줄 확인
3. 체크박스 **ON**

화면 참고: Changes — README.md + diff

모르는 파일이 많으면 README만 체크하고 나머지는 끄세요.

6. 메시지 쓰고 Commit

예시: README에 연습 한 줄 추가 · 5장 연습 커밋 · Update README

한 줄이면 충분. 비밀번호는 넣지 마세요.

1. Summary / Commit message에 입력
2. Description은 비워도 됨
3. **Commit to main**(또는 master / 현재 브랜치 이름) 클릭
4. Changes가 비면 성공

브랜치 이름이 main이든 master든 **그 버튼**이면 됩니다. 새 브랜치는 7장.

화면 참고: Summary + Commit to main

버튼이 회색이면 체크된 파일·메시지 글자를 확인.

이름/이메일 설정 창이 뜨면 github Username·가입 이메일이면 충분(실명 불필요).

7. 커밋만 됨 ≠ 웹에 반영

	커밋 직후	푸시 후(6장)
로컬 History	✓	✓
github.com	✗ 예전일 수 있음	✓
Desktop Push 배지	보여도 이 장에서는 누르지 않음	

확인: 웹 README에 연습 줄이 **없으면 정상**. Desktop History에는 새 커밋이 보일 수 있음.

화면 참고: Desktop History에 새 커밋 / 웹 README는 아직 예전

Push를 이미 눌렀으면 6장에서 웹 확인만 이어가면 됩니다.

미션

1. Desktop에서 클론 저장소 열기
2. README 한 줄 추가·저장
3. Changes 확인 · 체크 ON · 짧은 메시지 · **Commit**
4. **Push는 누르지 않음**

기본 경로: 로컬 수정 → Changes → 메시지 → Commit (푸시 없음)

이 장에서 안 함

푸시(6) · PR(7) · 새 브랜치 · 터미널 필수화 · amend·리셋 · Actions

짧게 정리

커밋 = 묶음 기록 · 스테이징 = 체크 · README 수정 → Commit · **푸시는 6장** · History에 내 커밋이 보이면 완주.

한글 메시지 OK. “커밋할 게 없다”면 저장·한 글자 더 수정.

화면 참고: 6장 예고 — *Push origin*

나두Ai · 『AI 하는 사람을 위한 깃허브 초간단』

6장. 푸시(Push) — GitHub에 올리기

5장 로컬 커밋을 Desktop **Push origin**으로 github.com에 올립니다.
웹에서 README·커밋을 확인합니다. **PR은 7장.**

학습 목표

- 푸시 = 내 컴퓨터 커밋을 GitHub 원격으로 보내기
- 웹에서 변경 확인 · 실패 시 로그인·권한·인터넷 · 클론 → 커밋 → 푸시를 말로

1. 푸시란?

푸시 = 내 컴퓨터의 커밋을 GitHub(원격)로 보내기

단계	어디에	
클론	내 PC	4장
커밋	내 PC 기록	5장
푸시	github.com	✓

Push origin 배지·버튼은 “올릴 커밋이 로컬에 있다”는 신호에 가깝습니다.

화면 참고: 푸시 비유 — PC History → github.com

이미 웹에 반영했다면 5장에서 Push를 눌렀을 수 있습니다. 이 장은 **웹 확인**까지가 목표입니다.

2. 시작 전

5장 Commit 완료 · Desktop 같은 계정 · 인터넷 · 저장소 URL을 열 수 있음.

커밋이 없으면 5장으로.

화면 참고: History에 커밋 + Push origin 배지

Push가 없거나 “0 to push”면 올릴 것이 없거나 이미 끝남 → README 한 글자 더 → Commit 후 재시도.

3. Push origin

1. Desktop에서 해당 저장소 선택
2. **Push origin / Push** / Push 1 commit to origin 클릭
3. 잠깐 대기 → 배지 숫자 사라지면 성공에 가까움

main/master 모두 Push면 됩니다. 새 브랜치·PR은 7장.

화면 참고: Push origin 버튼

여기서 막히면

Authentication → Desktop 계정 다시 Sign in.

Permission denied → 본인 저장소 · 같은 Username인지.

네트워크 → 연결 확인 후 재시도.

“Pull first” → 웹에서 같은 파일을 고친 뒤일 수 있음. 연습은 웹 편집을 피하고 다시 Push.

Pull 심화는 범위 밖(8장 방향만).

4. github.com에서 확인

1. <https://github.com/내Username/저장소이름> 새로그침
2. README에 5장 연습 한 줄
3. (선택) 최근 커밋 메시지가 5장 Summary와 같은지

4. Desktop Push 배지 없음

화면 참고: 웹 README + 최근 커밋 메시지

웹이 그대로면: 다른 저장소·Username인지 · 강력 새로그침.

커밋만 보이고 내용이 없으면 5장에서 README 체크·Commit을 되짚고 Commit→Push를 한 번 더.

5. 실패 시 먼저 볼 곳

순서	점검
1	Desktop·웹 같은 계정
2	쓰기 권한 있는 내 저장소
3	인터넷

6. 말로 한 바퀴

클론 — GitHub → 내 PC 복사

커밋 — 수정 후 로컬 기록

푸시 — 그 기록을 GitHub로 보내 웹과 같게

한 줄: 가져와 → 기록하고 → 올린다.

화면 참고: 클론 → 커밋 → 푸시 강조, PR은 흐리게

미션

Push origin → 오류 없이 종료 → 웹 새로그침 → README 확인 → **PR은 만들지 않음**

원하면 README 다른 한 줄 → Commit → Push → 새로그침을 한 바퀴 더.

이 장에서 안 함

PR·새 브랜치 · Pull/충돌 심화 · `git push` 필수화 · Actions · force push

짧게 정리

푸시 = 로컬 커밋을 GitHub로 · Push origin → 웹 확인 · 막히면 로그인→권한→인터넷 · **PR은 7장.**

Publish는 “새 원격을 처음 만들 때”에 가깝고, 4장 클론 경로에서는 보통 **Push. origin** ≈ “연결된 github.com 저장소”.

화면 참고: 7장 예고 — *Compare & pull request*

나두Ai · 『AI 하는 사람을 위한 깃허브 초간단』

7장. Pull Request(PR) — 변경을 제안하기

본인 저장소에서 짧은 브랜치 → 커밋·푸시 → **PR** → **Merge**까지 한 바퀴.

6장까지는 main(또는 master)에 바로 올리는 연습이었습니다. 이 장은 “옆 줄에서 고쳐 → 본줄기에 합쳐 주세요”.

다른 사람 저장소에 PR은 하지 않습니다. 혼자 연습이 가장 안전합니다.

학습 목표

- PR = 본줄기(기본 브랜치)에 합쳐 달라는 제안
- 브랜치 → 커밋·푸시 → PR · 제목·설명 짧게 · **Merge** = 제안이 합쳐짐

1. PR이란?

PR = 내 변경을 본줄기에 합쳐 달라고 제안하기

아직 합쳐진 상태가 아닙니다. Merge해야 본줄기에 들어갑니다.

단계	한 줄
브랜치	작업 줄 만들기
커밋·푸시	그 줄에서 고치고 올리기
PR	합쳐 달라는 제안
Merge	제안이 받아들여져 합쳐짐

한 바퀴의 마지막: 계정 → ... → 푸시 → **PR**

화면 참고: PR 비유 — 브랜치 → 제안 카드 → main

main에 바로 올리는 연습은 6장까지로 충분합니다. PR은 협업·검토 습관의 기본 버튼입니다.

2. 브랜치 (초보용)

브랜치 = 본줄기와 나란한 작업용 줄

- 본줄기: 보통 `main` 또는 `master`
- 작업 줄 예: `practice-pr`

이름 규칙·삭제·리베이스·충돌 해결은 다루지 않습니다. Desktop·웹에 보이는 기본 브랜치 이름을 따르면 됩니다.

화면 참고: *main + practice-pr 갈래*

3. 시작 전

클론→커밋→푸시 경험 · Desktop에 본인 저장소·같은 계정 · 인터넷.
푸시가 안 되면 6장부터.

화면 참고: *Desktop — 현재 브랜치 main*

4. 새 브랜치

1. 본인 저장소 선택 · 현재가 main/master인지 확인
2. **New branch** / Create branch
3. 이름: `practice-pr` (영문·숫자·하이픈 권장)
4. Create → 상단이 `practice-pr` 로 바뀌었는지 확인

여기서 막히면

New branch가 안 보이면 상단 브랜치 이름(Current branch)을 누르거나 **Branch → New**

Branch를 찾으세요. 이름은 영문·숫자·하이픈(`practice-pr`)을 권장합니다.

화면 참고: *New branch — practice-pr*

Current branch 클릭 또는 Branch → New Branch로 찾는 경우가 많습니다.

5. 그 위에서 고치고 → Commit → Push

5~6장과 같은 버튼. 다른 점은 브랜치가 `practice-pr` 인 것.

1. Show in Explorer/Finder → `README.md` 에 한 줄:

연습: 7장 PR용 한 줄

2. 저장 → Changes 확인 → 메시지 예: 7장 PR 연습 한 줄 추가

3. **Commit to practice-pr**

4. **Push origin / Publish branch** (새 브랜치 첫 업로드면 Publish에 가까움 — “원격에 올린다”면 같은 단계)

화면 참고: *Commit to practice-pr 후 Push / Publish*

“Commit to main”이면 아직 main → 4절로 돌아가 브랜치를 만드세요.

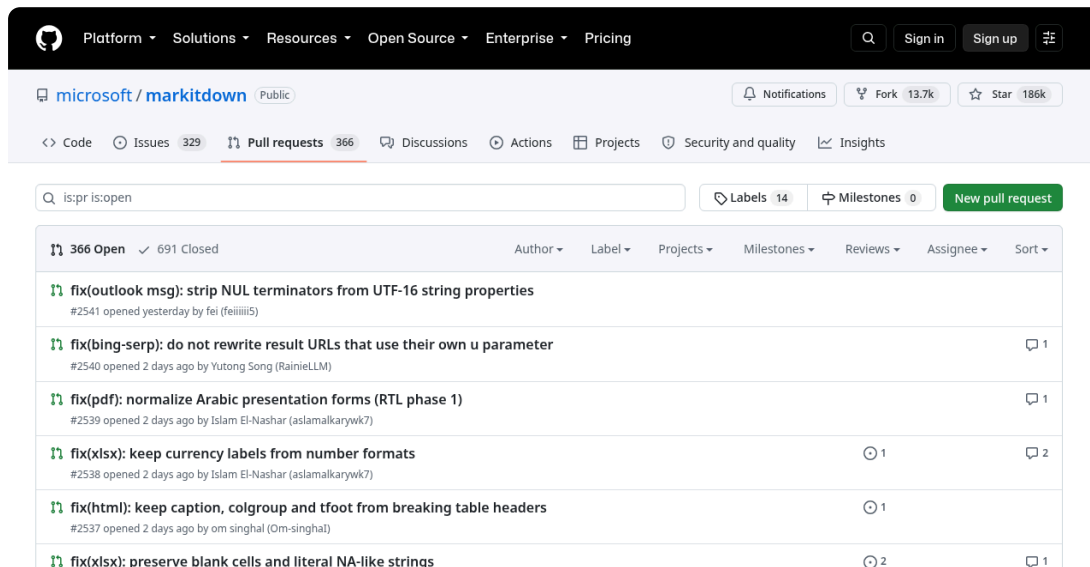
6. 웹에서 PR 열기

1. 본인 저장소 새로고침
2. **Compare & pull request** 배너가 있으면 클릭
3. 없으면 Pull requests → **New pull request**
 - **base:** main(또는 master) · **compare:** `practice-pr`
4. 제목·설명(7절) → **Create pull request**

아직 main에 합쳐진 것은 아닙니다.

여기서 막히면

Compare & pull request가 없으면 (1) Push/Publish 했는지 (2) 같은 저장소인지 (3) Pull requests → New pull request에서 base=main, compare= practice-pr 를 직접 고르세요. 충돌 경고면 새 브랜치+README 다른 한 줄로 다시 하는 편이 쉽습니다.



Pull requests 탭 예시(공개 저장소)

Pull requests 탭 예시(공개 저장소)

배너가 없으면: Publish 했는지 · 같은 저장소인지 · base/compare 직접 선택.

충돌 경고면 초보는 새 브랜치 + README 다른 한 줄로 다시. 충돌 해결은 범위 밖.

7. 제목·설명

제목 예: 7장 연습: README 한 줄 추가

설명 예: 연습용 PR. README 한 줄 추가. Merge까지 해 봅니다.

한글 OK. Draft는 필수가 아님 — 일반 Create면 됩니다.

화면 참고: Title·설명 + Create pull request

8. Merge

본인 저장소이므로 스스로 Merge해도 됩니다.

1. Merge pull request → Confirm merge
2. Merged 표시면 성공
3. (선택) Delete branch — 안 지워도 목표 달성

Merge = PR 제안이 받아들여져 본줄기에 합쳐진 것

화면 참고: Merge → Confirm → Merged

버튼이 비활성이면 본인 개인 연습 저장소인지 확인(회사 repo 비추천).

Merge 종류는 기본(merge commit)이면 충분. Squash·Rebase는 심화.

9. 합친 뒤 확인

Code 탭 · 브랜치 main · README에 연습 줄 · PR 상태 **Merged**.

(선택) Desktop에서 main으로 바꾼 뒤 Fetch/Pull — 강제하지 않음. 웹 README만 같아도 완료.

화면 참고: main README + Merged

말로 · 미션

브랜치에서 고쳐 올리고 → 합쳐 달라고 하고(PR) → 합친다(Merge).

미션: 본인 저장소 · practice-pr · README 한 줄 · Commit · Push/Publish · PR · Merge · main에서 확인 · 타인 저장소 PR 없음

이 장에서 안 함

타인 PR/Fork · 브랜치 전략 심화 · 충돌·rebase · Actions · 터미널 필수 · force push

기본: Desktop 브랜치 → Commit → Push → 웹 PR → Merge → README 확인

짧게 정리

PR = 제안 · Merge = 합쳐짐 · 내 저장소에서 혼자 연습.

다음 8장은 자주 막히는 장면 Q&A.

화면 참고: 8장 예고 — Q&A

나두Ai · 『AI 하는 사람을 위한 깃허브 초간단』

8장. 자주 막히는 장면 Q&A

새 기능을 배우지 않습니다. 2~7장에서 막힐 때 **먼저 볼 곳**만 짚습니다.

본인 계정·본인 저장소 기준. 무단 접근·우회는 다루지 않습니다.

쓰는 법: 가까운 Q → “먼저 볼 곳” 위부터 → 안 되면 아래 복구 순서 → 범위 밖 표.

화면 참고: 8장 지도 — 계정 / 어디에 / 클론 / 푸시 / PR / 범위 밖

A. 계정 · 로그인 · 권한

Q1. 비밀번호가 틀리다고 나옵니다

1. `https://github.com` 인지
2. Sign in의 Username/email이 가입 때와 같은지
3. 비밀번호 재입력(대소문자·한영)
4. **Forgot password?** 로 본인 이메일 재설정만
5. Sign out → Sign in 한 번 더

비밀번호·복구 메일은 **본인만** 다릅니다.

Q2. 이메일 인증이 안 끝납니다

메일함(스팸 포함) → Verify/Confirm → 만료 시 재발송 → 다른 브라우저.
공식 재설정만. “해킹”류 조언은 책 밖입니다.

Q3. Private이 안 보이거나 푸시가 거절됩니다

1. 브라우저·Desktop **같은 본인 계정**
2. URL의 Username이 내 것인지
3. Private은 로그아웃·다른 계정 창에서 안 보일 수 있음
4. 그래도 안 되면: 로그인 → 권한 → 인터넷 (6장)

Q4. Desktop과 브라우저 계정이 다릅니다

브라우저 Username vs Desktop Accounts 비교 → 다르면 Desktop Sign out 후 본인으로 Sign in → 저장소 다시 열기(필요 시 재클론).

화면 참고: 브라우저 Username vs Desktop Accounts

B. “어디에 올렸지?”

Q5. 커밋했는데 웹에 없습니다

커밋 ≠ 푸시. History에만 있으면 **Push** → 웹 새로고침 → URL·계정 확인.

Q6. 저장소·폴더가 헷갈립니다

웹 주소 `github.com/Username/저장소` ↔ Desktop **Local path**를 한 줄로 짝 지으세요.

엉뚱한 폴더를 고치고 있으면 Desktop에서 맞는 저장소를 다시 엽니다.

화면 참고: URL + Local path 나란히

Q7. 고쳤는데 변경이 없습니다

저장 여부 · Desktop이 그 저장소를 보는지 · 이미 Push까지면 웹과 같아 변경 없음 = 정상일 수 있음.

C. 클론

Q8. Clone URL·버튼을 모르겠습니다

저장소 → **Code** → **HTTPS** → 복사 또는 Open with Desktop → Local path 확인 → Clone.
SSH·터미널 심화는 필수 아님.

Q9. destination already exists

같은 이름 폴더가 Local path에 있음 → 다른 상위 경로 또는 이미 있는 저장소를 Desktop에서 열기. 기존 폴더를 함부로 삭제하지 마세요.

Q10. 클론했는데 README가 없습니다

웹에 README가 있는지(3장) · 같은 저장소인지 · Fetch/Pull은 선택(충돌은 범위 밖).

D. 커밋 · 푸시

Q11. Commit이 회색입니다

Changes 체크 ON · 메시지 한 줄. 이미 커밋 끝났으면 다음은 Push.

Q12. Push 실패 (auth / permission / network)

로그인 → 권한 → 인터넷. force push·토큰을 남에게 묻는 방법은 안 함.

Q13. Push할 커밋이 없습니다

Commit 먼저이거나, 이미 올려서 웹과 같은 상태일 수 있음 → 웹 새로고침으로 확인.

E. PR

Q14. Compare & pull request가 안 보입니다

작업 브랜치인지 · Push/Publish 했는지 · 이미 열린 PR인지 · 없으면 README 한 줄 더 → Commit → Push 후 새로고침.

Q15. Merge가 안 됩니다

Open 상태인지 · Merge → Confirm 두 단계 · 충돌·Actions 검사는 범위 밖 · 본인 repo인지 (7장).

Q16. 잘못된 PR을 열었습니다

합치지 않을 거면 Close. 필요하면 새 브랜치로 7장 미션 다시. 이미 Merge면 다음 연습으로 (되돌리기 심화는 밖).

F. 영문 · 범위 밖

비슷한 문구	뜻
Sign in / out	로그인 / 로그아웃
Clone / Commit / Push	복사 / 기록 / 보내기
Pull Request / Merge	제안 / 합치기
Permission / Authentication	권한 / 인증
Conflict	같은 줄 충돌 (심화)

주제	이 책	다음
Actions	×	공식 Docs — 9장에서 “나중”
rebase·force push	×	심화 Git
충돌 해결	×	merge conflict 입문
SSH·토큰 심화	×	HTTPS + Desktop 유지
타인 PR / Fork	×	7장은 내 repo
VS Code Git 심화	×	Desktop 한 바퀴 먼저

복구 순서 (최소)

계정: github.com · 브라우저 Username · Desktop 같은 계정 · Private은 로그인 창

어디에: Push 했나 · 새로고침 · URL · Local path 짝 · 파일 내용

동작: 클론 Code→HTTPS · 커밋 체크+메시지 · 푸시 로그인·권한·인터넷 · PR 브랜치
→Push→Compare

증상별 복습: 가입 2 · 저장소 3 · 클론 4 · 커밋만 5 · 웹 반영 6 · PR 7 · 막힘 8 · 미션 9

짧게 정리

막히면 새 기능보다 **점검 순서** · 브라우저=Desktop 계정 · 안 보이면 **커밋 vs 푸시** · 영문은 표
단어만 · Actions·충돌·force push는 나중에.

다음 **9장**: 혼자 미니 미션 · 한 바퀴 복습 · 나중에 볼 주제(이름만).

화면 참고: 9장 예고 — 미니 미션

나두Ai · 『AI 하는 사람을 위한 깃허브 초간단』

9장. 다음에 연습할 것

본편 한 바퀴를 끝냈습니다. 새 기능을 깊게 배우지 않습니다.

혼자 돌릴 **미니 미션**, **한 바퀴 더** 점검, “나중에 볼 주제” 이름만, 그리고 **부록 두 개** 메뉴입니다.

연습 저장소에 실명·실제 이메일·API 키를 넣지 마세요. 이름은 `practice-loop-01` 처럼 평범하게.

학습 목표

미션 3~5개 고르기 · 계정→...→PR 복습 · Issues·브랜치 이름은 힌트만 · Actions·심화 Git은 나중에 · 부록 A·B를 다음 메뉴로

1. 미니 미션 (오늘 3개면 충분)

A. README 한 줄 — 고치기 → 커밋 → 푸시 → 웹 확인 (5~6장)

B. 새 연습 저장소 — `practice-loop-01` · Private + README ON → 클론 → 한 줄 → 커밋 → 푸시 (3~6장)

C. PR 한 번 더 — 본인 repo · 짧은 브랜치 → 커밋·푸시 → PR → 본인 Merge (7장) · **타인 repo PR 금지**

D. “어디에 올렸지?” — 커밋 메시지 · Push 여부 · 같은 URL · 안 되면 8장 (커밋≠푸시)

E. (선택) NOTES.md — 막힌 점 한 줄(키·비번 금지) → 커밋 → 푸시

화면 참고: 미션 A~E — 오늘 고른 3개

막히면 8장 증상별 표로. 새 버튼보다 먼저 볼 곳.

2. 한 바퀴 더

☐ 계정 로그인 (2)

- ☐ 저장소 하나 (3)
- ☐ 클론 · 경로 얹 (4)
- ☐ 커밋 · 커밋 ≠ 푸시 (5)
- ☐ 푸시 · 웹에서 같음 (6)
- ☐ PR · 본인 repo에서 Merge 한 번 (7)
- ☐ 막히면 8장 순서

계정 → 저장소 → 클론 → 커밋 → 푸시 → PR

화면 참고: 한 바퀴 여섯 칸

3. 나중에 — 이름만

주제	한 줄	이 책
브랜치 이름	짧고 역할이 보이게	7장 최소
Issues	할 일·버그 메모 게시판	이름만
Fork	남의 repo 내 계정 복사	필수 아님
Actions	푸시 후 자동 검사·배포	나중에
심화 Git	rebase·force push 등	나중에

Issues·브랜치 이름 규칙 연습은 이 장에서 하지 않습니다. 팀 규칙은 나중에.

4. 일부러 나중에

Actions · rebase/cherry-pick/force push · SSH·토큰 심화 · 타인 PR·모노레포 — 한 바퀴 (계정~PR)가 먼저.

이 책 성공 기준은 Actions가 아닙니다.

5. 다음 메뉴 — 부록

부록 A 부록-나두인기영상실습.md — 나두 영상 주제 ↔ 깃허브 한 바퀴 (Lab 1~8). 영상에서 본 흐름을 내 저장소에 남기기.

부록 B 부록-깃허브트렌드실습.md — Trending 후보로 같은 한 바퀴 (Lab 1~8).

고르는 법: 영상 보고 왔으면 A 하나 · 트렌드로 연습이면 B 하나 · 둘 다면 A 후 B(하루에 하나만도 OK).

시크릿·실명 규칙은 부록 안내를 따릅니다.

화면 참고: 부록 A / B 표지

본편 마무리

1. 혼자 반복 · 미션 3개면 충분
2. 복습: 계정→저장소→클론→커밋→푸시→PR
3. Issues·브랜치 이름은 힌트만 · Actions·심화는 미룸
4. 다음은 부록 실습 · 막히면 8장

고맙습니다. 본편 한 바퀴를 끝까지 따라와 주셔서.

화면 참고: 본편 완주 — 한 바퀴 + 부록 A/B

나두Ai · 『AI 하는 사람을 위한 깃허브 초간단』

부록 A. 나두 인기 영상 — 따라 하기 실습

순수 설치·실행·클릭 튜토리얼. 나두Ai(@nadooai) 인기 영상 툴·사이트를 열고 → 쓰고 → (선택) 깃허브에 짧게 저장.

본편 1~9장 = 계정·저장소·클론·커밋·푸시 참고. 이 부록 = 영상 툴 실습장. 무료 이북.

UI가 바뀌어도 비슷한 버튼이면 됩니다. 주소는 시청한 영상·공식 안내만 — 책에 없는 URL을 지어 내지 마세요. 표기는 항상 영상에서 본 툴 이름 .

쓰는 법

1. Lab 하나 고르기 → 해당 영상 틀기 → 툴·사이트 1개만.
2. 따라하기 번호대로: 연다 → 입력/클릭 → 결과 확인.
3. (선택) 아래 공통 저장으로 lab0N/NOTE.md 만 남기기.

공통 저장소 예: my-ai-lab · 폴더 lab01 ... lab08 . Lab별 저장소(lab01-free-ai 등)로 나뉘도 됨.

시크릿 — 인트로에서 한 번만

해도 됨	하지 말 것
API_KEY= 자리표시	sk-실제키... 커밋
.env.example 에 키 이름만	.env 통째 푸시
.gitignore 에 .env	스크린샷에 키 전체
실수 시 즉시 키 폐기	“나중에 지울게” 방치

개인 실명·직장 메일·얼굴은 README·캡처에 넣지 마세요.

결과 저장 — 공통 (Lab 끝에서 재사용)

1. <https://github.com> 로그인 → + → **New repository** (이미 `my-ai-lab` 이면 그곳으로).
2. 새 저장소면: name `my-ai-lab` · **Private** · **Add a README ON** · **Create repository**.
3. **Add file** → **Create new file** → 예: `lab01/NOTE.md` → 메모 붙여 넣기 → **Commit changes** (`lab01: note`).
4. Desktop이면: 로컬 저장 → Changes → Commit → **Push origin** (본편 5~6장).

Lab 끝에서는 「본편처럼 커밋·푸시」 한 줄만 보면 됩니다.

NOTE.md 최소 칸 (복붙)

Lab:
영상:
툴: 영상에서 본 툴 이름
링크: (확인한 공식 주소만)
한 일(클릭·명령 요약):
결과(요약·로컬 경로):
시크릿 포함 여부: 없음

공통 패턴

- 웹 AI: 주소 → 로그인 → 입력 → 전송 → 복사.
- 다운로드 앱: 공식 받기 → 설치 → New → 저장.
- 웹툰: 열기 → 검색/업로드/변환 → 결과.
- 깃허브 OSS: README Quick start → (선택) Fork/Clone → 첫 명령만.

실습 목차 (조회수 = 제공값만)

Lab	영상(조회수)	따라 할 일	저장 파일 예
1	이 AI들, 공짜로 씁니다 (~9434)	무료 AI 1개 · 프롬프트 1회	<code>lab01/NOTE.md</code>
2	영상 제작비 0원 만드는 법 (~9175)	영상/음성/자막 툴 · 초안 1개	<code>lab02/NOTE.md</code>

Lab	영상(조회수)	따라 할 일	저장 파일 예
3	이 사이트들, 즐겨찾기 필수입니다 (~5676)	사이트 1개 · 오늘 일 1칸	lab03/NOTE.md
4	매달 나가던 구독료, 끊으세요 (~5076)	무료·OSS 대안 1개 · 실행/데모	lab04/NOTE.md
5	도커 하나로 홈서버 완성 (~2576)	Compose/실행 · 로컬 1회	lab05/NOTE.md
6	요즘 다 이걸로 씁니다 (~1808)	추천 툴·방식 · 채팅/ 코드 1회	lab06/NOTE.md
7	PDF·사진·전송, 이제 안 사도 됩니다 (~1074)	PDF·이미지·전송 중 1회	lab07/NOTE.md
8	이 사이트들, 있는지도 몰랐죠? (~563)	숨은 툴 · AI 보조 1회	lab08/NOTE.md

Lab 1. 「이 AI들, 공짜로 씁니다」 (~9434)

목표: 영상 속 무료(또는 무료 한도) AI **1개**에 내 문장을 넣어 결과 **1회**.

준비

- 브라우저 · 영상 「이 AI들, 공짜로 씁니다」
- 영상에서 본 툴 이름 **1개**
- 넣을 **내 문장 1개** (키·비밀번호·타인 민감정보 제외)

따라하기

1. 일시정지 후 오늘 쓸 도구 이름 적기: 영상에서 본 툴 이름 .
2. 영상·공식 주소로 열기. (검색 광고 링크 금지)
3. **Log in / Sign up / 시작하기 / Get started** 등이 있으면 본인 계정으로만.
4. 채팅창·프롬프트 칸· Ask / Message 입력란 찾기.
5. 아래 틀을 붙여 넣고 원문 만 교체.

역할: (예: 3줄 요약 도우미 / 존댓말 메일 초안 / 코드 원인 후보)

원문:

(여기에 내 글·메모·코드 일부)

부탁: (한 줄로 구체적으로)

제약: 한국어, 모르는 것은 추측이라고 표시

6. **Send / Submit / Generate / 전송.**
7. 답 전체 선택 → 복사 → 메모장에 붙여 넣기.
8. 메모 상단: 툴: · 링크: · 결과요약: (3~5줄; 공개 저장소면 요약만).
9. 남은 전화번호·메일·키는 *** 로.
10. (선택) 문장만 바꿔 한 번 더 — 기록은 1회분이면 충분.

부탁 한 줄 예: 아래 메모를 고객 메일 초안, 존댓말 8문장 이내. · 아래 에러 기준 원인 후보 2개와 확인할 파일. · 불릿을 결정사항/할 일/미정으로만. · 얼굴·실명 없이 20초 슷폼 혹 5개.

팁: 결제·업그레이드 창은 **닫고**, 영상에서 말한 무료 한도·모드만.

끝: lab01/NOTE.md · 메시지 lab01: free AI once · (선택) .gitignore 에 .env / *.key ·
본편처럼 커밋·푸시.

Lab 2. 「영상 제작비 0원 만드는 법」 (~9175)

목표: 무료 영상·음성·자막 툴 1개로 15~30초용 초안 1개.

준비

- 영상 시청 · 주제 한 줄(예: 커밋과 푸시 차이 20초) · 목표 길이
- 영상에서 본 툴 이름 — 역할: 영상 / 음성 / 자막 중 하나

따라하기

1. 칸 하나: 대본만 / TTS 음성만 / 자막만 / 짧은 편집만 .
2. 맞는 툴 이름 적고 공식 주소·설치 페이지 열기.
3. 웹: 열기 → **New project / Create / 시작**. 설치형: 공식 다운로드 → Next → 실행 → **New**.
4. 대본을 메모장에 먼저 (다른 AI 초안 OK).

혹(1문장):

본론(2~3문장):

끝(1문장):

5. 소리 내어 읽고 15~30초에 맞게 줄이기.
6. 툴에 대본 넣기 — 에디터: 텍스트/캡션 트랙 · TTS: 붙여 넣기 → **Generate** → 저장 · 자막: 줄 단위 타임라인.
7. **미리보기 / Preview / Play**.
8. 가능하면 **Export / 내보내기 / Render**(또는 프로젝트 저장).
9. 경로 적기. 예: Downloads/lab02-draft .
10. 메모: 툴 · 역할 · 스크립트 · 길이·해상도·음성 · 로컬 경로.

팁: 큰 MP4/WAV는 깃에 올리지 않음 — 경로·스크립트만. BGM은 영상 안내 무료 소스만, 불확실하면 무음.

끝: lab02/NOTE.md · lab02: draft note · 바이너리 Add 금지 · 본편처럼 커밋·푸시.

Lab 3. 「이 사이트들, 즐겨찾기 필수입니다」 (~5676)

목표: 영상 속 사이트 **1개**로 오늘 할 일 **1칸** 처리.

준비

- 영상 · 오늘 할 일 1개 · 영상에서 본 툴 이름 · 확인한 공식 주소

따라하기

1. 할 일과 겹치는 사이트 하나. 메모: 사이트: ... / 일: ____.
2. 공식 주소를 주소창에 입력해 열기.
3. 쿠키·동의는 필수만 처리.
4. 가입 필수면 본인 메일 Sign up. 아니면 게스트·검색만.
5. 유형별:
 - **검색:** 키워드 → Enter → 결과 제목 3개 복사
 - **생성·조합:** 옵션/팔레트/템플릿 → **Apply / Generate / Download**
 - **업로드: Upload** → 개인정보 없는 샘플만 → 변환·다운로드
 - **문서:** 해당 페이지 → 필요 문단·표만 복사
6. 결과 위치 확인(다운로드 파일명 또는 메모 텍스트).
7. (선택) AI에 결과 붙여 아래를 표로 정리. 기준: ____ 1회.

팁: 로그인·지역 제한이면 영상 속 **다른 사이트 1개**로 1~6 반복.

끝: lab03/NOTE.md (사이트·링크·오늘 일·클릭/입력·결과) · lab03: site once · **본편처럼 커밋·푸시.**

Lab 4. 「매달 나가던 구독료, 끊으세요」 (~5076)

목표: 유료 대신 무료·OSS 대안 **1개**를 열어 같은 종류 입력으로 **1회** (웹 데모 또는 로컬).

준비

- 영상 · 대체할 루틴 한 줄 · 영상에서 본 툴 이름 또는 공개 깃허브 주소
- (선택) Desktop · 터미널 · 연습용 입력(실명·카드·사내 로그 금지)

따라하기

1. 대안 이름·주소 메모. 출처 = 영상·공식·깃허브만.
2. 그 페이지 열기.
3. 깃허브면 README **Getting started / Quick start / Install**만 스크롤.
4. (선택) **Fork** → Create fork.
5. **A 또는 B** 중 하나만.

A. 웹 데모·호스팅

1. **Demo / Live / Try / App** 링크 열기.
2. 연습용 문장·파일 입력.
3. **Run / Submit / Generate**.
4. 메모: 입력종류 / 결과한줄 / 에러문(키·이메일 가림) .

B. 클론 후 로컬

1. **Code** → **HTTPS** → URL 복사.
2. Desktop: **File** → **Clone repository** → URL → Local path → Clone. 또는 `git clone <URL>` (본편 4장).
3. README 첫 **설치 명령**만 실행 (`npm i` · `pip install -r requirements.txt` 등 문서에 있는 것만).

4. 키 필요 시 로컬 `.env` 에 실값. 저장소에는:

```
# .env.example
API_KEY=
```

5. `.gitignore` 에 `.env` 확인·추가. (`grep -n "\.env" .gitignore` 또는 웹에서 파일 열어 추가·커밋)
6. README 실행 명령으로 기동 → 연습 입력 1회.
7. `git status` 에 `.env` 가 **안** 뜨는지 확인. 뜨면 Add하지 말고 ignore 수정.
8. 공통 메모: 이전: ___ / 대안: ... / 경로: A또는B / 명령·클릭: ___ / 결과또는에러: ___ .

팁: 설치가 깨져도 가린 에러+시도 명령을 `NOTE.md` 에. 카드 해지·환불은 다루지 않음.

끝: `lab04/NOTE.md` (+ `.env.example` 만) · (선택) Issue `lab04: blocked` · `lab04: alt trial` ·
본편처럼 커밋·푸시.

Lab 5. 「도커 하나로 홈서버 완성」 (~2576)

목표: 홈서버·셀프호스트 예시 **1개**의 Docker/Compose를 따라 **localhost**로 열거나, 같은 문서로 실행 메모 완성.

준비

- 영상 · 목적 한 줄 · 영상에서 본 툴 이름 · 공식 깃허브/문서 URL
- PC 여유 · Docker 설치 여부 · 빈 데이터 폴더(중요 원본 금지)

따라하기

1. 후보: 영상에서 본 툴 이름 .
2. 공식에서 Docker / Docker Compose / Quick start 열기.
3. 예시 `compose.yaml` / `docker-compose.yml` 준비. 예시 비밀번호·토큰은 연습용으로 바꾸고 깃에 넣지 않음.
 - ports 예 "8080:80" → 브라우저는 `http://localhost:8080` .
 - volumes 는 단계 4의 빈 폴더만.
4. Docker 있으면:
 1. `mkdir ~/nadoo-lab05 && cd ~/nadoo-lab05`
 2. `mkdir ~/nadoo-lab05-data`
 3. 문서대로 compose 저장.
 4. 문서 기동 명령 그대로 (예: `docker compose up -d`).
 5. 포트 보이면 `http://localhost:포트` .
 6. UI 열리면 테스트 메모·파일 1개만.
 7. 종료 절의 명령(`docker compose down` 등)을 메모에 복사.
5. Docker 없으면: 명령·이미지·포트·볼륨을 NOTE.md 에 옮기고 Docker 설치 후 재시도 한 줄. 강제 설치 없음.
6. 캡처 시 비밀번호·토큰·집 주소 자르기.

7. 포트포워딩·HTTPS·외부 공개 IP **설정하지 않음** — 로컬만.

팁: port is already allocated 면 ports 번호를 바꾸거나 컨테이너 내리고 메모만 커밋.

끝: lab05/NOTE.md (후보·명령·포트·테스트·종료, 시크릿 없음) · lab05: compose notes ·
본편처럼 커밋·푸시.

Lab 6. 「요즘 다 이걸로 씁니다」 (~1808)

목표: “요즘 쓴다”고 한 툴·모델·방식 **1개**를 채팅 또는 코드에 **1회** 적용.

준비

- 영상 · 영상에서 본 툴 이름 · AI 채팅 또는 코드 편집기 · 작은 작업 1개

따라하기

1. 툴/방식: ... / 진입: 사이트|앱|모델선택|플러그인|프롬프트템플릿 .
2. 영상 화면까지:
 - **웹:** 주소 → 로그인 → 모델/모드 드롭다운에서 동일 항목
 - **앱:** 설치·실행 → 설정에서 옵션 ON
 - **플러그인:** 마켓에서 검색 → Install → 활성화
 - **템플릿:** 프롬프트·스니펫 불러오기
3. 평소와 같은 작은 작업문 입력.
4. **Send / Run / 적용** 1회.
5. 결과 복사 → 메모.
6. 하단:

전: (평소 방식 한 줄)
후: (이번 툴·방식 한 줄)

7. 링크·모델명·옵션명은 화면 그대로 (별명만 쓰지 말 것).

팁: 유료만 열리면 무료 한도·모델만. 비교는 동일 질문으로 옛 1번·새 1번만.

끝: lab06/NOTE.md · lab06: try once · 본편처럼 커밋·푸시.

Lab 7. 「PDF·사진·전송, 이제 안 사도 됩니다」 (~1074)

목표: 무료·OSS 1개로 PDF·이미지·전송 중 하나를 더미로 1회.

준비

- 영상 · 종류 1택: PDF / 사진 / 전송 · 영상에서 본 툴 이름
- 더미 파일 1개 (주민번호·계약서·신분증 스캔 금지)

따라하기

1. 종류: PDF|사진|전송 / 툴:
2. 웹이면 오픈 · 설치형이면 공식 설치 · OSS면 README 실행 절.
3. 파일 선택 / Open / Upload / 열기 찾기.
4. 더미 없으면: PDF= lab07 dummy 인쇄→PDF · 사진=풍경·물건(얼굴·신분증 제외) · 전송=위 중 하나(가능하면 zip).
5. 파일 넣기.
6. 영상 강조 기능 **하나만**: 합치기·분할·압축·리사이즈·포맷 변환·임시 링크 등.
7. **Download / Save / 복사 링크** → 경로·URL 메모.
8. 전송형: 만료·비밀번호 확인 후 테스트 뒤 **만료·삭제** 즉시.
9. 다운로드·클라우드에 민감 파일 섞임 여부 파일명만 훑기.
10. 메모: 툴 · 클릭 순서(3~6줄) · 결과 경로 · 더미만 사용: 예 .

팁: 웹 업로드가 불안하면 로컬·오프라인만. 경로 오류 시 Desktop/lab07-dummy.pdf 로 옮겨 재선택.

끝: lab07/NOTE.md 만(결과 바이너리 깃 금지) · lab07: file once · **본편처럼 커밋·푸시.**

Lab 8. 「이 사이트들, 있는지도 몰랐죠?」 (~563)

목표: 잘 안 알려진 툴 **1개**로 재료 1개 만들고 AI에 1회 넣기.

준비

- 영상 · 작은 작업 1개 · 영상에서 본 툴 이름 · 공식 링크 · AI 채팅(또는 메모장)

따라하기

1. “처음 알았거나 잊고 있던” 항목 1개.
2. 공식 링크로 열기. 비회원 가능하면 가입 없이.
3. 핵심 입력란·버튼 찾기.
4. **검색:** 키워드 → 제목·요지 3개 · **변환:** 입력 → **Convert** → 복사 · **정리:** 템플릿·표에 내 항목 3칸.
5. AI 옆에 두고 재료 붙여 넣기.
6. 지시 1회:

아래 재료를 바탕으로 초보용 설명 5문장을 써 줘.
재료:
(붙여 넣기)

7. AI 답 요약 3줄.
8. 카탈로그 한 줄: 정확한 이름 · 한 줄 용도 · 링크 · **오늘사용: YES** .

팁: 접속 불가·지역 제한이면 영상 속 **다른 1개**로 1~8 반복.

끝: lab08/NOTE.md (표: 이름|용도|링크|사용 + 적용 메모) · lab08: hidden tool · **본편처럼 커밋**
·푸시.

마치며

- 커밋·푸시가 어색하면 본편 5~6장. 다음: 부록 B.
- 조회수는 제공값. 제목이 조금 달라도 **같은 주제 Lab**. 주소는 영상·공식만.

나두Ai · 부록 A · 따라 하기 · 무료 이북

부록 B. 깃허브 공개 저장소 설치 따라 하기

예시 레포(교체 가능). Trending 날짜에 묶이지 않음 — AI 초보가 설치해 보기 좋은 공개 저장소 예시.

나두Ai · 『AI 하는 사람을 위한 깃허브 초간단』 · 따라 하기식 설치·실행 · 무료 이북.

전제: 본편 1~5장(가능하면 6장). 흐름은 **설치(또는 다운로드) → 한 번 실행 → 결과 파일을 내 깃허브에 짧게.**

이 부록을 쓰는 법

1. 목표 읽기
2. 공식 URL 열기
3. 번호 스텝대로 설치·실행
4. 결과 파일을 내 저장소에 커밋

- 명령은 공식 **README**가 바뀌면 README 우선. 이 책은 초보용 한 줄 예시.
- Star만 누르고 끝내지 마세요. **설치·실행 결과 파일**이면 충분.

시크릿 — 인트로에서 한 번만

올리면 안 됨	올려도 됨
<code>.env</code> 실값, API 키, 토큰	<code>.env.example</code> (키 이름만)
신분증·계약서·회사 문서	연습용 가짜 텍스트·요약
실명·직장 메일 스크린샷	가린 화면 설명 한 줄

결과 저장 — 공통 (한 번만 만들어도 됨)

1. `https://github.com/new` → `nadoo-install-lab` (또는 Lab마다 `nadoo-lab-1` ...) · **Private** · **Add a README ON** · Create.
2. Lab 폴더 예: `lab-01/` ... `lab-07/` .

3. **Add file** → **Create new file** → Lab이 정한 결과 파일(키·실문서 제외) → **Commit** (Lab-N: install note).
4. Desktop: 파일 저장 → Changes → Commit → Push (본편 5~6장).

Lab 끝에서는 「본편처럼 커밋·푸시」 한 줄만 보면 됩니다.

Python · README가 다를 때

Lab 1·5·6·7은 Python 필요 → python.org 후 터미널 새로 열기. Lab 2·4는 Python 없이 가능. 빨간 글자면 이 책 예시를 고집하지 말고 해당 README Installation / Usage를 복사.

Lab별 남길 파일

Lab	올리면 좋은 것	올리면 안 되는 것
1	NOTE.md , 연습용 out.md 일부	실제 기밀 PDF
2	NOTE.md (프롬프트 제목만)	AI 대화 전문·개인정보
3	NOTE.md (설치·허용 소스만)	받은 동영상 파일
4	NOTE.md (gh 버전)	gh 토큰
5	hello_openai.py , .env.example , .gitignore	.env
6	NOTE.md	HF 토큰·대용량 가중치
7	app.py , NOTE.md	불필요

실습 목차 (가벼운 7 Lab)

Lab	저장소	한 줄로 하는 일
1	microsoft/markitdown	pip 설치 → 파일 1개 Markdown 변환
2	f/awesome-chatgpt-prompts	클론/다운로드 → 프롬프트 1개 복사해 AI에 사용
3	yt-dlp/yt-dlp	설치 → 허용된 샘플 URL 1개 받기

Lab	저장소	한 줄로 하는 일
4	cli/cli	GitHub CLI 설치 → <code>gh --version</code>
5	openai/openai-python	pip → hello 스크립트 뼈대 (키는 <code>.env</code>)
6	huggingface/huggingface_hub	pip → CLI로 아주 작은 파일 받기(또는 로그인 문서만)
7	gradio-app/gradio	pip → 5줄 hello UI

이번 목록에서 제외 (Skip)

무거운 설치·계정·Docker·에이전트 하네스는 **따라 하기 A**에서 뺌.

항목	처리	이유
paperless-ngx/paperless-ngx	제외	Docker·문서함 기동 부담
n8n-io/n8n	제외	워크플로·의존성 무거움
affaan-m/ECC	제외	에이전트 하네스·설계 중심
BuilderIO/agent-native	제외	에이전틱 앱·매핑 속제형
anthropics/claude-code	제외	계정·설치 선택 부담
Open-Dev-Society/OpenStock	제외	실행/환경이 가벼운 설치 튜토리얼에 덜 맞춤
mihail911/modern-software-dev-assignments	제외	과제·학습 계획형 (설치 실습 아님)
higgsfield-ai/higgsfield 등 GPU ML	제외	초보 설치 범위 밖
xtekky/gpt4free	제외	스케치·약관 리스크

레포가 없거나 README가 크게 바뀌면 **같은 난이도 공개 저장소**로 교체해도 됨. 흐름(설치 → 실행 1번 → 커밋)만 유지.

Lab 1. Markdown — microsoft/markitdown

목표: 연습용 파일 1개를 Markdown으로 바꾸고, 변환 결과(또는 설치 메모)를 내 깃허브에 남긴다.

공식 URL: <https://github.com/microsoft/markitdown>

설치·실행

1. Python 확인 (3.10+ 권장):

```
python3 --version
```

Windows에서 안 되면 `python --version`.

2. (권장) 가상환경:

```
python -m venv .venv  
source .venv/bin/activate
```

Windows 활성화는 공식 README 따름.

3. 설치:

```
pip install 'markitdown[all]'
```

4. **연습용** PDF/docx/txt만. 신분증·계약서·회사 기밀 금지.

5. 변환:

```
markitdown 샘플.pdf -o out.md
```

또는:

```
markitdown 샘플.pdf > out.md
```

6. `out.md` 를 열어 글자가 보이면 성공.

7. PDF 의존성 오류면 README extras → `pip install 'markdown[pdf]'` 등.

화면 참고: 터미널 *markdown 실행* · *out.md 일부*

끝: `NOTE.md` + (연습용이면) `out.md` 일부 · `lab-1: markdown convert` · **본편처럼 커밋·푸시.**

Lab 2. Awesome ChatGPT Prompts —

f/awesome-chatgpt-prompts

목표: 프롬프트 1개를 골라 AI에 붙여 넣고, 사용한 프롬프트 **제목만** 내 깃허브에. (무거운 설치 없음)

공식 URL: <https://github.com/f/awesome-chatgpt-prompts>

설치·실행

1. 위 URL 열기.
2. **Code** → **Download ZIP** 또는:

```
git clone https://github.com/f/awesome-chatgpt-prompts.git
```

3. README·목록에서 **프롬프트 1개**만.
4. AI 채팅에 붙여 넣고 **짧은 질문 1개**.
5. 답변 전문·개인 대화는 올리지 않음.

화면 참고: 저장소 README · 프롬프트 하나 선택

끝: NOTE.md 에 날짜 · 프롬프트 제목 · 사용한 AI · 결과 한 줄 · lab-2: one prompt trial ·
본편처럼 커밋·푸시.

Lab 3. yt-dlp — yt-dlp/yt-dlp

목표: yt-dlp 설치 후 **저작권·이용이 허용된** 짧은 샘플 URL 1개만 받기.

저작권: 본인 영상·퍼블릭 도메인·다운로드 **명시 허용** 자료만. 남의 유료·저작권 콘텐츠 무단 금지.

연습만이면 직접 찍은 클립을 로컬에 두고 “허용 URL로 1회 받음”만 NOTE.md 에 적어도 됨.

공식 URL: <https://github.com/yt-dlp/yt-dlp>

설치·실행

1. README **Installation** — OS별 원라이너 우선.
2. Python 있으면:

```
pip install -U yt-dlp
```

Windows winget 등이 README에 있으면 그대로.

3. 버전:

```
yt-dlp --version
```

4. **허용된** 짧은 URL 하나 (본인이 고름):

```
yt-dlp "https://예시-허용-URL"
```

5. 파일이 보이면 성공. 큰 파일·불법 소스는 올리지 않음.

화면 참고: `yt-dlp --version` · 다운로드 완료 한 줄

끝: NOTE.md (설치 한 줄 · 허용 소스 1회 · 동영상 파일 커밋 금지) · lab-3: yt-dlp install note · 본편처럼 커밋·푸시.

Lab 4. GitHub CLI — cli/cli

목표: `gh` 설치 → `gh --version`. (로그인·클론은 선택)

공식 URL: <https://github.com/cli/cli>

설치·실행

1. README **Installation**에서 내 OS.
2. 예 (환경마다 다름 — README 우선):

- macOS: `brew install gh`
- Windows: `winget install --id GitHub.cli`
- Linux: 배포판별 안내

3. 새 터미널:

```
gh --version
```

버전 숫자가 보이면 본미션 완료.

4. (선택) 계정 연동:

```
gh auth login
```

강제 아님. 부담되면 버전만.

5. (선택) 내 저장소 클론:

```
gh repo clone 내계정/내저장소
```

화면 참고: `gh --version` 출력

끝: NOTE.md 에 OS · 설치 방법 · `gh` 버전만(토큰 금지) · `lab-4: gh version` · **본편처럼 커밋·푸시.**

Lab 5. OpenAI Python — openai/openai-python

이 Lab만 **API 키·과금** 개념. 키는 `.env` 에만. 부담되면 Lab 1₄.6₇ 먼저.

목표: 패키지 설치 + 키는 `.env` 에만 두는 hello **빠대**. 키는 커밋하지 않음.

공식 URL: <https://github.com/openai/openai-python>

실제 API 호출은 **본인 계정·요금·약관** 확인 후. 키 없으면 빠대 + `.env.example` 만으로도 완료.

설치·실행

1. 가상환경(권장) 후:

```
pip install openai python-dotenv
```

2. `.gitignore` 에:

```
.env
```

3. `.env.example` (커밋용 — **가짜 자리만**):

```
OPENAI_API_KEY=CHANGE_ME
```

4. 로컬 전용 `.env` (커밋 금지):

```
OPENAI_API_KEY=여기에_본인_키
```

5. `hello_openai.py` **빠대**:

```

import os
from dotenv import load_dotenv

load_dotenv()
key = os.getenv("OPENAI_API_KEY")

if not key or key == "CHANGE_ME":
    print("키가 없습니다. .env만 로컬에 준비하고 커밋하지 마세요.")
else:
    # 공식 README의 최신 클라이언트 예제를 여기에 붙입니다.
    print("키 로드됨(값은 출력하지 마세요). README 예제로 한 번만 호출해 보세요.")

```

6. 실행:

```
python hello_openai.py
```

키를 `print` 하지 마세요. Chat/Responses 예제는 **공식 문서** 복사.

화면 참고: `.gitignore`에 `.env` · `hello` 스크립트

끝: `hello_openai.py` · `.env.example` · `.gitignore` · `NOTE.md` · **`.env` 금지** · `lab-5: openai`
`hello skeleton` · **본편처럼 커밋·푸시.**

Lab 6. Hugging Face Hub —

huggingface/huggingface_hub

목표: `huggingface_hub` 설치 후 CLI로 **아주 작은** 공개 파일 받기, 또는 로그인 문서만 읽고 메모.

공식 URL: https://github.com/huggingface/huggingface_hub

설치·실행

1. 설치:

```
pip install -U huggingface_hub
```

2. CLI:

```
hf --help
```

3. 경로 A — 작은 공개 파일

README/ `hf download` 예시로 **용량 작은** 공개 파일 하나 (이름·모델은 Hub 기준 본인 선택):

```
hf download 작은공개파일/또는모델 --include "*.txt" --local-dir ./hf-sample
```

명령이 바뀌었으면 공식 README 그대로.

4. 경로 B — 받기 부담

README 로그인(`hf login`)만 읽고 오늘은 로그인 안 해도 됨. NOTE.md 에 “문서만 확인”.

5. 토큰·캐시 비밀 파일은 커밋하지 않음.

화면 참고: `hf --help` 일부

끝: NOTE.md (설치 · A: 파일 이름만 / B: 문서만 · 토큰·가중치 금지) · `lab-6: huggingface_hub`
note · **본편처럼 커밋·푸시.**

Lab 7. Gradio — `gradio-app/gradio`

목표: Gradio 설치 → 5줄 안팎 hello UI를 로컬에서 한 번.

공식 URL: <https://github.com/gradio-app/gradio>

설치·실행

1. Python 3.10+ 후:

```
pip install --upgrade gradio
```

2. `app.py` :

```
import gradio as gr

def greet(name):
    return "Hello " + name + "!"

demo = gr.Interface(fn=greet, inputs="text", outputs="text")
demo.launch()
```

(공식 README 예제가 더 최신이면 README 우선.)

3. 실행:

```
python app.py
```

4. 터미널 주소(예: `http://127.0.0.1:7860`)를 브라우저로.
5. 이름 넣고 Submit → 인사 보이면 성공.
6. 끝: `Ctrl+C` . `share=True` 는 초보 Lab에서 생략해도 됨.

화면 참고: 로컬 Gradio 인사 화면(실명 가림)

끝: `app.py` + `NOTE.md` (“로컬 launch 성공”) · `lab-7: gradio hello` · **본편처럼 커밋·푸시.**

레포를 바꿀 때

예시 7개는 고정 커리큘럼이 아님. 아래만 지키면 다른 공개 저장소로 교체 가능.

1. 초보가 몇 명령 또는 브라우저 몇 클릭으로 끝낼 것
2. 공식 README에 Installation / Usage
3. GPU·Docker·유료 계정 강제 없을 것 (있으면 Skip 표 + 대체)
4. 결과로 NOTE.md 또는 작은 코드/변환 파일 1개

교체 후보 예: `psf/requests` (pip hello), `sharkdp/bat`, `BurntSushi/ripgrep`. 본문 예시는 위 7개 기준.

부록 B를 끝낸 뒤

1. `nadoo-lab-1` ... 7 (또는 `lab-0N/`)에 NOTE·결과 파일이 보이는지.
2. 7개 전부 불필요 — **하나라도** 설치→실행→커밋이면 습관 성공.
3. 레포가 사라지거나 무거우면 같은 난이도로 교체, 흐름만 유지.
4. 부록 A와 주제 겹치면 같은 날 둘 다 할 필요 없음.
5. 본편 8장 Q&A · 9장으로 “한 바퀴” 이어 가기.

공개 저장소는 구경거리가 아니라, **설치해 보고 결과 파일만 내 깃허브에 남기는 연습 재료.**

고지 · 법적 주의

※ 이 문서는 안내 체크리스트이며 법률 의견이 아닙니다.

『AI 하는 사람을 위한 깃허브 초간단』 (나두Ai)을 이용하기 전에 한 번 읽어 주세요.

1. 저작권 · 이용 범위

- 이 전자책 PDF의 저작권은 **나두Ai**(익명 발행)에 있습니다.
 - 무료로 제공하며, **개인 학습용**으로 이용할 수 있습니다.
 - **무단 재판매·유료 재배포·타인 명의의 유료 상품화**는 금지합니다.
 - 공식 배포 주소(<https://tinalinkeom.vercel.app/ebook>)를 공유하는 것은 허용합니다.
-

2. 제3자 상표 · 비제휴

- GitHub, TikTok 및 각종 AI·오픈소스 명칭은 각 권리자의 상표일 수 있습니다.
 - 이 전자책·나두Ai는 GitHub 등의 **공식 교육·제휴·인증 커리큘럼이 아닙니다**.
 - 화면 문구는 서비스 업데이트로 바뀔 수 있으니, 비슷한 위치·비슷한 문구를 기준으로 따라가 주세요.
-

3. 제3자 코드 · 저장소

- 실습은 공개 저장소 링크·클론·포크·설치 안내입니다. 이 PDF가 타인의 소스코드를 상품으로 재판매하지 않습니다.
 - 각 저장소의 **라이선스(MIT, Apache 등)**는 독자가 직접 확인·준수해야 합니다.
 - 예시로 든 저장소는 시점에 따라 삭제·변경될 수 있습니다.
-

4. 나두Ai 콘텐츠

- TikTok·YouTube 등 나두Ai 영상은 별도 작품입니다.
- 이 전자책은 **따라하기 가이드**이며, 영상 파일 자체를 포함·재배포하지 않습니다.
- 영상은 삭제되거나 노출이 바뀔 수 있습니다.

5. 교육 목적 면책

- 학습·실습 안내입니다. 특정 결과·수익·취업·보안 완벽을 보장하지 않습니다.
- 도구·API·요금제·화면은 변경될 수 있습니다.
- API 키·비밀번호·`.env` 등 비밀값이 유출되지 않도록 관리할 책임은 독자에게 있습니다. 커밋·공유 전에 한 번 더 확인하세요.

6. 금지 안내

불법 스크래핑, 서비스 약관 위반, 무단 침해, 악성 자동화 등에 이 자료를 사용하지 마세요.

7. 제공 방식 · 문의

항목	내용
브랜드	나두Ai
상품	『AI 하는 사람을 위한 깃허브 초간단』 PDF (무료)
받는 곳	https://tinalinkeom.vercel.app/ebook
직접 파일	https://tinalinkeom.vercel.app/guides/nadoo-github-guide.pdf
문의	나두Ai 채널(@nadooai) 및 https://tinalinkeom.vercel.app

현재는 **무료** 제공됩니다. 유료 판매·환불 규정은 적용하지 않습니다.

나두Ai